Camunda 8 - Process Communication

What you will learn

This course provides detailed, hands-on experience with horizontal process communication in Camunda. You will learn how to address typical project needs by understanding, recognizing, and using different approaches that use process communication.

At the end of this course, you will be able to:

- Recognize different process communication approaches
  - Call activities
  - Signal events
  - Message events
- Implement, run, and test message events
  - In your process definitions
  - In your application logic
- Recognize different cancellation approaches
  - Message boundary events
  - Event-based gateways
  - Event subprocesses
- Implement, run, and test call activities
- Implement, run, and test signal events

Business requirement

A fictitious organization, the Camundanzia Insurance Company, is used in the scenario for this course

Camundanzia

Scenario

The Camundanzia insurance company would like to continue introducing process automation throughout its organization.

Camundanzia would like to enhance its existing **Business Travel** process with **automated communication** using other processes. The current version of the process no longer meets the business requirements, as it only consists of manual tasks performed by employees.

Currently, customers ask Camundanzia consultants for technical on-site support. Before calling the customer to confirm the trip, the consultant must call the travel agency to provide the necessary travel information (date, flight, and hotel) and wait for the booking confirmation. While waiting for the confirmation, the consultant examines the information about the intervention and prepares for the journey.

To manage the booking of all trip requirements, including transportation tickets, hotel reservations, etc., the improved version of the process will need to connect to the **Travel Agency** process.

The booking request will be reviewed, and the **Travel Agency** process will report any problems that come up throughout the booking process. If problems arise, the business trip will be cancelled.

In order to complete the travel process, the consultant must provide input regarding the business trip and specify whether **any expenses should be reimbursed**.

Also, the **Travel Policy** rules may be modified in the future. If so, then the employee must be notified immediately.

User story

These four **consultant user stories** describe the requirements:

User story

- As a consultant, I want to provide my destination, hotel, and transport for the business trip so that I can receive my booked tickets without a manual interaction with the travel agency.

User story

- As a consultant, I want to be notified if my trip is cancelled so that I can call the customer to cancel my visit.

User story

- As a consultant, I want to request reimbursement for any travel-related costs so that I can get compensated.

User story

- As a consultant, I want to receive updates whenever changes occur to the **Travel Policy** so that I can prepare for my trip accordingly.

You will use a Minimum Viable Product (MVP) approach to develop an application that meets these requirements.

Acceptance criteria

The initial acceptance criteria are as follows:

1. The booking **request is automatically sent** to the travel agency.
2. The **booking confirmation is automatically sent** to the consultant.
3. Any **cancellation of a booking is communicated** to the consultant.
4. Any required **expense reimbursement can be claimed** during a business trip.
5. Any changes to the **Travel Policy** are communicated to the consultant.

Iterative development

Iterative development

Camunda 8 is well suited to an iterative development approach; applications can be easily revised and upgraded without affecting in-flight processes.

Outline solution

You now have a clear set of requirements and acceptance criteria for the **Camundanzia Business Travel** process.

The sections below provide an overview of how to develop an **interprocess communication** solution that meets these requirements.

Camundanzia

Scenario

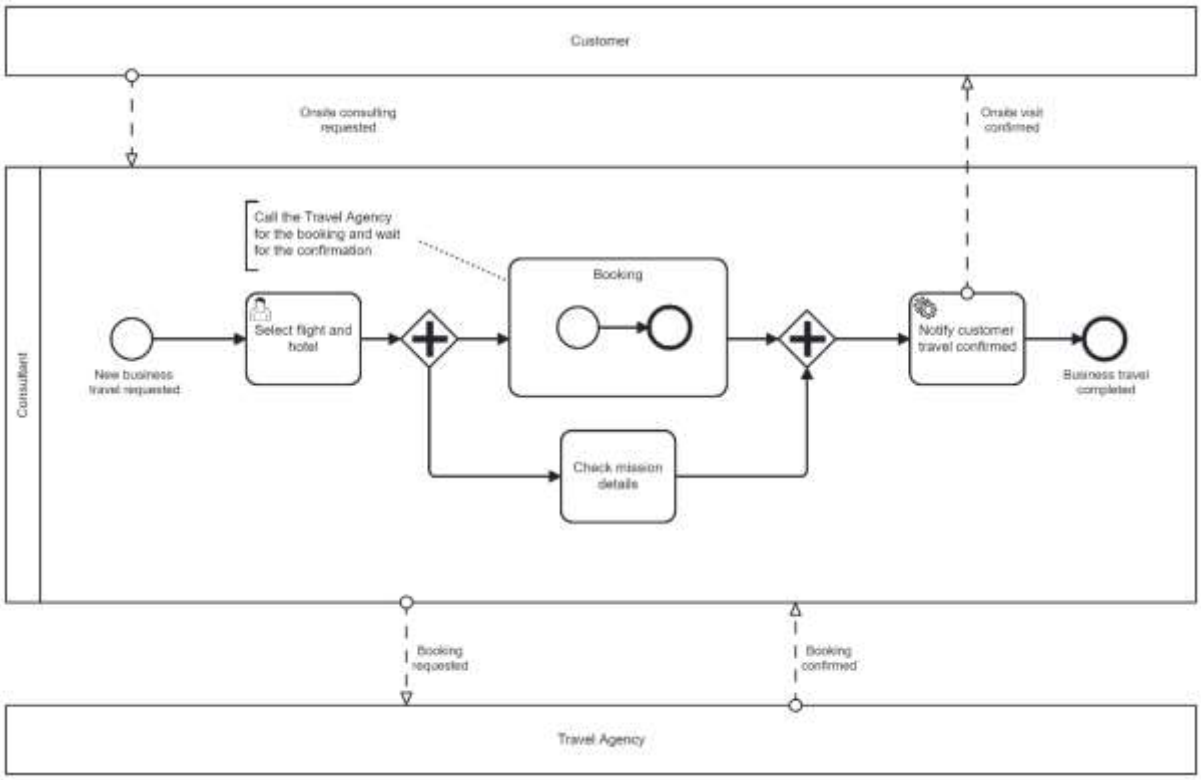
Imagine that you have recently joined Camundanzia as a new developer and have been assigned to the Camunda 8 project team.

Your first assignment is to enhance the **Business Travel** process using the requirements from the previous lesson.

Fortunately for you, a more experienced Camunda developer has drafted an outline solution for you. Your task is to implement a solution based on the outline.

Business Travel process

Take a look at the current **Business Travel** process:



Current Business Travel process

In this process:

1. The **customer** requests an on-site consultation.
2. The assigned **consultant** starts the **Business Travel** process. They:

    - Select the flight and the hotel according to the date and the customer's site.
    - Call the travel agency to book the desired flight and hotel.
    - Wait for the booking confirmation.
    - Check the mission details while the booking confirmation arrives.
3. Once the booking is confirmed, the **customer** receives confirmation of the on-site visit and the **consultant** is ready for the mission.
4. The **Business Travel** process is **completed**.

Outline solution

For each of the acceptance criteria that have been identified, the more experienced developer has provided an overview of the work that must be completed.

Project review

In order to start working on the process improvements, you must first understand the project structure and the **Travel Agency** process.

Review the project

Clone the git repository to your local computer and checkout the structure.

Project setup

Processes need to be deployed into your cluster (you will need to provide your cluster credentials in the application). Once you finish, you will need to test the connection and the project.

Deploy and test the project

Deploy the Business Travel process, the Reimbursement Claim process, and the Travel Agency process. Then, test the connection to your cluster.

Communicate with the Travel Agency process

The Business Travel process has to communicate with the Travel Agency process to start a new booking request, and to notify it that the booking was completed.

Send a message to the Travel Agency process

Send a message to start a new booking request.

Catch a confirmation message from the Travel Agency process

Catch a message to confirm that the booking was completed.

Cancel the Business Travel process

If the booking is cancelled, the Travel Agency process has to communicate that to the Business Travel process.

Catch a cancellation message from the Travel Agency process

Catch a message to confirm the booking was cancelled.

Communicate with the Reimbursement Claim process

For the final part of the process, if any expenses during a business trip need to be reimbursed, then the consultant will call the Reimbursement Claim process.

Call Reimbursement Claim process

Call the Reimbursement Claim process using a call activity.

Notify if Travel Policy changed

Whenever the travel terms and conditions change, the consultant must be informed.

Notify if Travel Policy changed

Send a signal to the Booking subprocess to notify the consultant if the Travel Policy has changed.

Development plan

Now that you have identified the steps to implement your solution, you can create a plan for the work that needs to be performed and the tools that are required.

- **Create a Camunda 8 SaaS account** *(optional)*
  - Provides access to the Modeler
  - Allows you to create a cluster
- **Create a cluster** *(optional)*
  - Provides access to the Zeebe engine
  - Provides access to Operate
  - Provides access to Tasklist
- **Create a project**
  - Performed from the Web Modeler
- **Import the initial processes: Business Travel, Travel Agency, and Reimbursement Claim**
  - Performed from the Web Modeler
- **Deploy the initial processes: Business Travel, Travel Agency, and Reimbursement Claim**
  - Performed from the Web Modeler
- **Test a Business Travel process instance**
  - Performed from your preferred IDE, Tasklist, and Operate
- **Add and configure a send task in the Booking subprocess**
  - Performed from the Web Modeler
- **Throw a BPMN message from a job worker**
  - Performed from your preferred IDE
- **Catch a BPMN message in the Booking subprocess**
  - Performed from the Web Modeler
- **Catch a BPMN message in the Booking Cancellation event subprocess**
  - Performed from the Web Modeler
- **Call the Reimbursement Claim process using a call activity**
  - Performed from the Web Modeler
- **Throw a BPMN signal**
  - Performed from the Zeebe API (CLI client)

Proceed to the next lesson when you are ready to get started.

Review existing project

Project

Download the course project ☞ here.

Overview

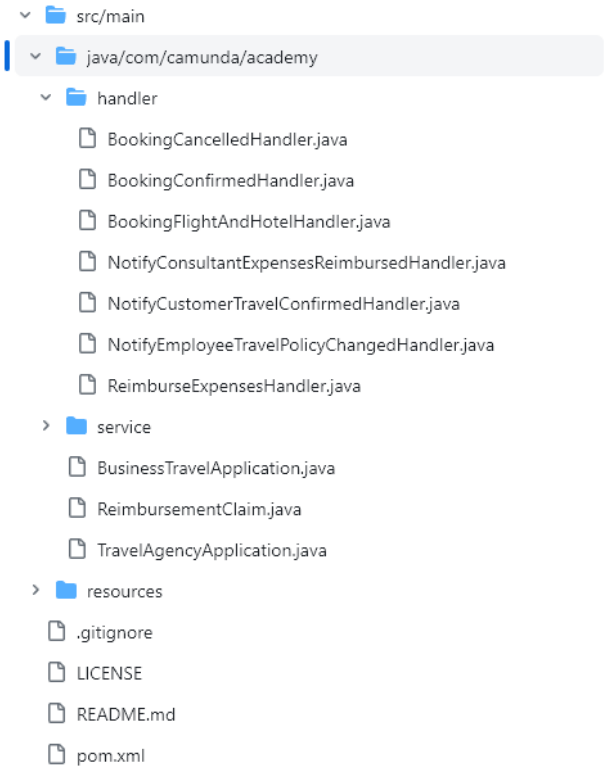
The aim of this course is to focus on process communication, so most of the content is provided. Your goal is to improve existing processes for the following scenario:

- A customer of your company asks for an on-site consultation.
- Once the request arrives, the consultant identifies the customer's location, and then checks for available flights and lodging.
- Your company is working with a travel agency that recently shared a booking process. You will improve your existing process to communicate with them, automating the consultant process as much as you can.
- You can assume that in a real use case scenario, this travel agency process and logic would exist in the **Travel Agency** cluster. However, you incorporated it into the logic of your **Business Travel** application in a separate **TravelAgencyApplication** main class, along with a separate **ReimbursementClaim** main class.

Review the application structure

Application

The application is located in `src/main/java/com/camunda/academy`.



Business Travel application structure

The application consists of:

- The **BusinessTravelApplication** class, which contains the main method and starts the workers for the **Business Travel** process.
- The **TravelAgencyApplication** class, which contains the main method and starts the workers for the **Travel Agency** process.
- The **ReimbursementClaim** class, which contains the main method and starts the workers for the **Reimbursement Claim** process.
- The **handlers** package, which contains different worker handlers:
  - The **NotifyCustomerTravelConfirmedHandler** handles the customer notification when the travel is confirmed.
  - The **NotifyEmployeeTravelPolicyChangedHandler** handles the consultant notification when the **Travel Policy** has changed.
  - The **BookingFlightAndHotelHandler**, **BookingConfirmedHandler**, and the **BookingCancelledHandler** handle the **Travel Agency** logic.
  - The **NotifyConsultantExpensesReimbursedHandler**, and the **ReimburseExpensesHandler** handle the **Reimbursement Claim** logic.
- The **service**, which contains all services that are called by the workers. Typically these are the ones you do not want to execute in your tests as they do not include the glue code for the process, but rather the business logic.

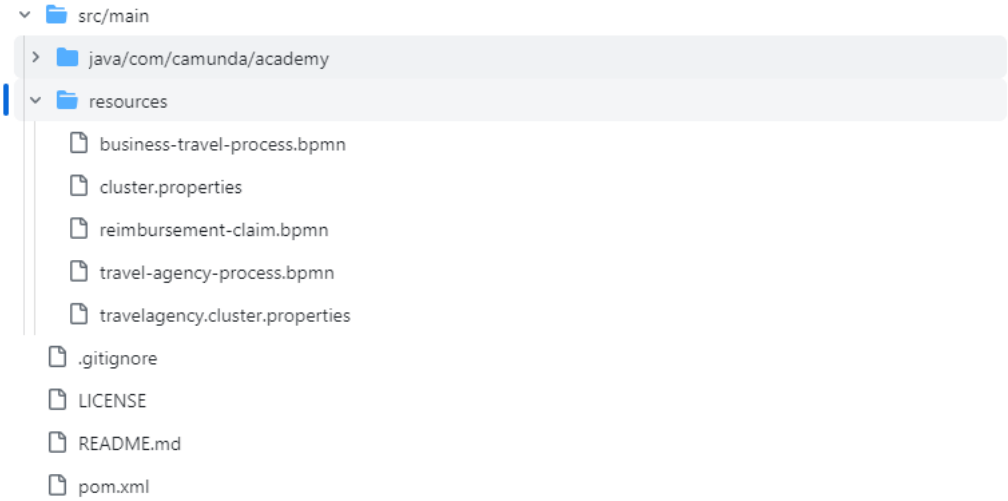
Travel Agency and Reimbursement Claim logic

For the purposes of this course, the **Travel Agency** logic and the **Reimbursement Claim** logic are included in the same **Business Travel** application.

Review the process definitions

Process definition

The process definitions are located in `src/main/resources`.



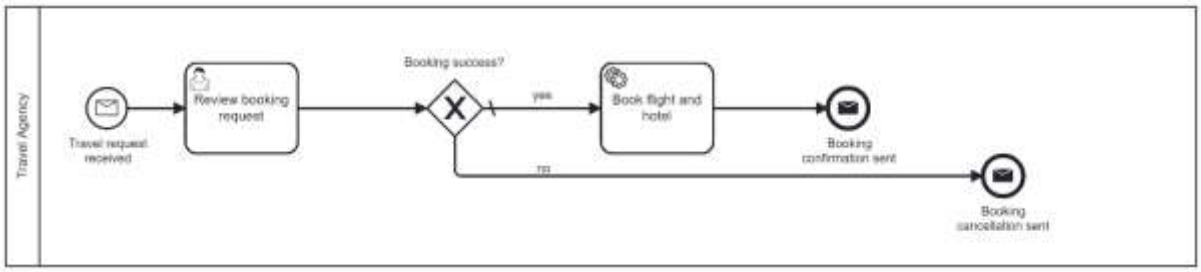
Business Travel application resources

Travel Agency process

Take a look at the **Travel Agency** process.

Info

This process is provided; you don't need to modify it. It represents a simplified version of a more complex process and defines the events and activities you will use to enhance the Business Travel process.



Travel Agency process

The process consists of:

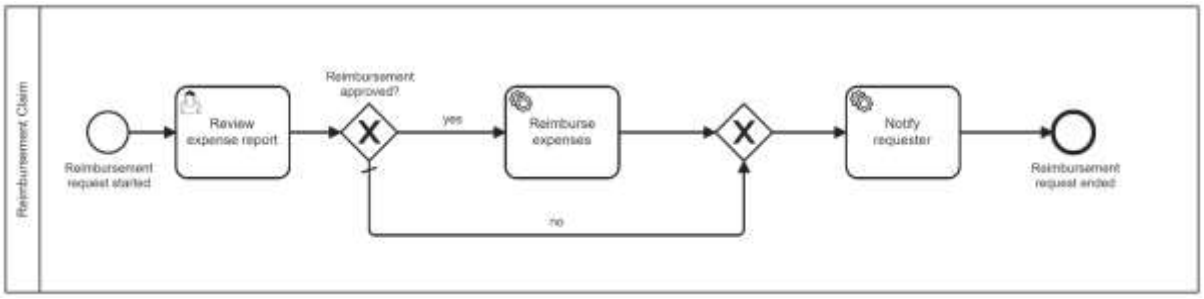
- A message start event (**Travel request received**)
- A user task (**Review booking request**)
- An exclusive gateway to check whether the booking was a success:
    - If successful:
        - A service task (**Book flight and hotel**)
        - A message end event (**Booking confirmation sent**)
    - If not successful:
        - A message end event (**Booking cancellation sent**)

Reimbursement Claim process

Take a look at the **Reimbursement Claim** process.

Info

This process is provided; you don't need to modify it. It represents a simplified version of a more complex process and defines the events and activities you will use to enhance the Business Travel process.



Reimbursement Claim process

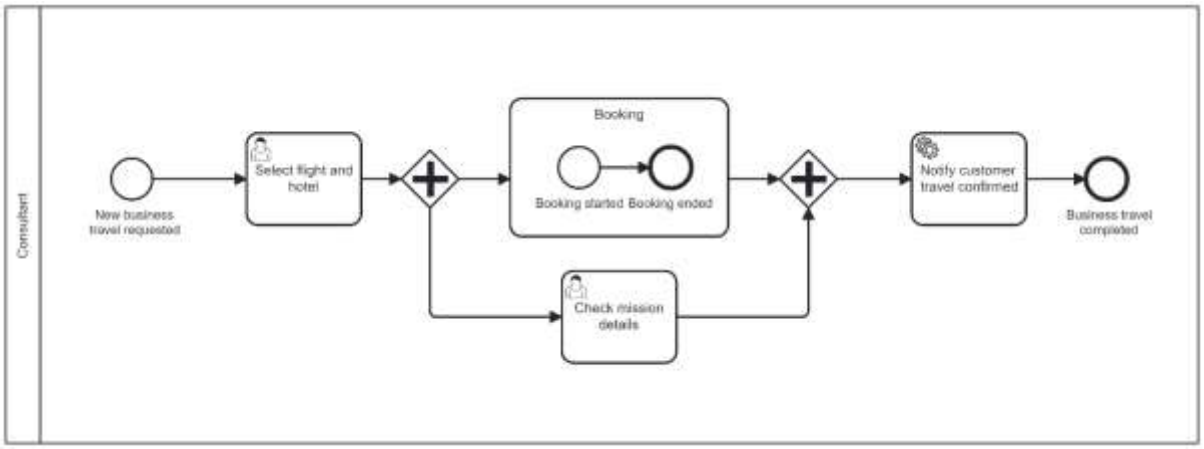
The process consists of:

- A start event (**Reimbursement request started**)
- A user task (**Review expense report**)
- An exclusive gateway to check whether the reimbursement was approved.
    - If approved, then a service task (**Reimburse expenses**) is completed.
- A service task (**Notify requester**)
- An end event (**Reimbursement request ended**)

Business Travel process

Take a look at the **Business Travel** process.

As explained in the previous lesson, this is your starting point: the process that you will enhance during the course. You will start by focusing on the **Booking** subprocess.



Business Travel process

Review the cluster credentials

Cluster credentials

The cluster credentials are located in `src/main/resources`.

The exercises in this course are designed to emulate a use case scenario in which interprocess communication occurs between processes in two different clusters.

- The main cluster credentials are set in the `cluster.properties` file.
- The second cluster credentials are set in the `travelagency.cluster.properties` file.

For the purpose of this course, you can set the same credentials of your single cluster in both files without issue.

Info

If you have multiple credentials, you can set different credentials for your clusters if you prefer. In that case, you will need to deploy the **Travel Agency** process to the second cluster.

Review the project

Congratulations! You successfully cloned the git repository to your local computer and checked the structure.

Prepare environment

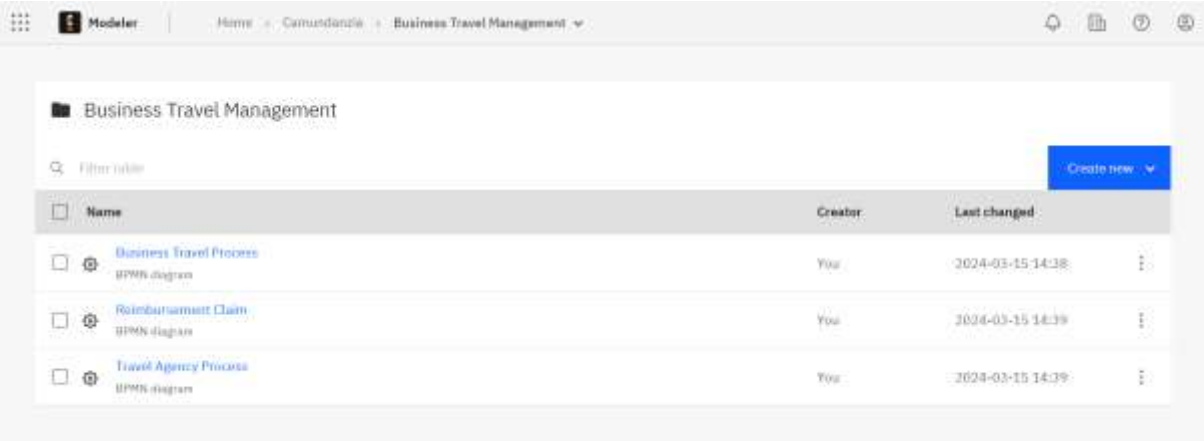
Create a new project

1. Navigate to the Camunda 8 Web Modeler.
2. Open your **Camundanzia** project.
3. Create a new folder (**Create new** > **Folder**) and call it "Business Travel Management".

Upload the files

1. Upload the process definitions to the **Business Travel Management** folder (click **Create new** > **Upload files**).
2. Select all of the files (`Business Travel Process.bpmn`, `Travel Agency Process.bpmn`, and `Reimbursement Claim.bpmn`) and then click **Open**.

Your files should look like this:



Process definitions with updated names

Deploy the processes

1. Navigate to the **Business Travel** process and deploy it to your main cluster.
2. Navigate to the **Reimbursement Claim** process and deploy it to your main cluster.
3. Navigate to the **Travel Agency** process and deploy it to your main cluster.

Multiple clusters

If you are following this course using a multiple cluster approach, deploy the **Travel Agency** process in the **second cluster**.

Import the Business Travel project

1. Import the downloaded project into a Java integrated development environment (IDE) such as Eclipse, NetBeans, IntelliJ, etc.

Set up the cluster credentials

1. Set up your main cluster credentials using the `resources/cluster.properties` file.

```
ZEEBE_ADDRESS = XXX.dsm-1.zeebe.camunda.io:443
ZEEBE_CLIENT_ID = XXX
ZEEBE_CLIENT_SECRET = XXX
ZEEBE_AUTHORIZATION_SERVER_URL = https://login.cloud.camunda.io/oauth/token
ZEEBE_TOKEN_AUDIENCE = zeebe.camunda.io
```

2. Use the same credentials to set up the `resources/travelagency.cluster.properties` file.

Multiple clusters

As mentioned before, if you are following this course using the multiple clusters approach, set up the second cluster credentials into this `travelagency.cluster.properties` file.

Run and test

1. Run `BusinesTravelApplication.java`.
2. Check the IDE console to view the last printed messages:

```
Connected to Cluster 1: TopologyImpl{brokers=[BrokerInfoImpl{nodeId=1,
host='zeebe-1.zeebe-broker-service.xxx-zeebe.svc.cluster.local', port=26501,
version=8.5.0, partitions=[PartitionInfoImpl{partitionId=1, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=2, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=3, role=LEADER,
health=HEALTHY}]}, BrokerInfoImpl{nodeId=0, host='zeebe-0.zeebe-broker-
service.xxx-zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]},
BrokerInfoImpl{nodeId=2, host='zeebe-2.zeebe-broker-service.xxx-
zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]}],
clusterSize=3, partitionsCount=3, replicationFactor=3, gatewayVersion=8.5.0}
```

3. Run a new Business Travel Process instance from the Camunda **Modeler** using the following JSON as an input:

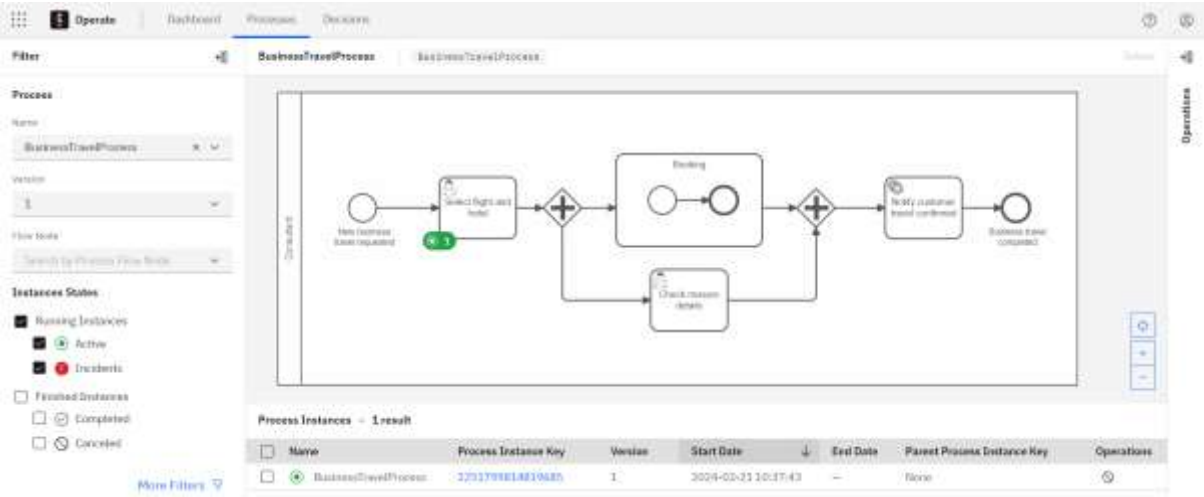
```
{
  "travelDate": "2024-11-06",
  "travelDestination": "Paris",
  "travelFlight": "LH 1074",
  "travelHotel": "Hotel Park Lane",
```

```
  "travelRequestId": "1695286596244"
}
```

Travel Request Identifier

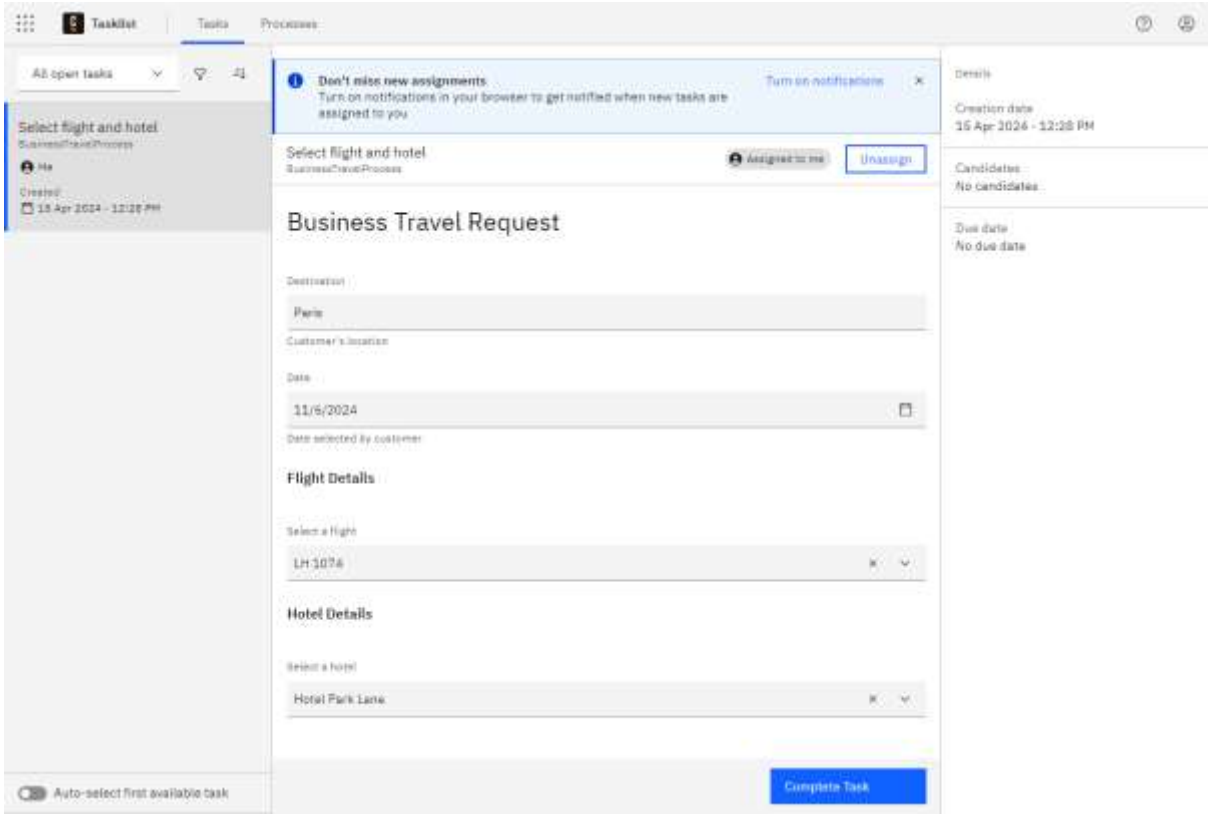
Note that `travelRequestId` is the unique identifier of each process instance. Remember to use a different identifier to test the process with multiple instances. It will be used as a correlation key.

4. Navigate to Camunda 8 **Operate**.
5. Notice if there is a running instance of the **Business Travel** process.
6. When the instance arrives at the **Select flight and hotel** user task, proceed to the next step.



Select flight and hotel user task

7. Navigate to Camunda 8 **Tasklist**.
8. Assign the **Select flight and hotel** user task to yourself and complete it.



Business Travel Request form

9. When the next user task (**Check mission details**) appears, assign it to yourself and complete it.
10. Navigate to the IDE console and review the last printed messages:

```
Connected to Cluster 1: TopologyImpl{brokers=[BrokerInfoImpl{nodeId=1,
host='zeebe-1.zeebe-broker-service.xxx-zeebe.svc.cluster.local', port=26501,
version=8.5.0, partitions=[PartitionInfoImpl{partitionId=1, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=2, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=3, role=LEADER,
health=HEALTHY}]}, BrokerInfoImpl{nodeId=0, host='zeebe-0.zeebe-broker-
service.xxx-zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]},
BrokerInfoImpl{nodeId=2, host='zeebe-2.zeebe-broker-service.xxx-
zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]}],
clusterSize=3, partitionsCount=3, replicationFactor=3, gatewayVersion=8.5.0}
1695286596244 Business Travel confirmed
```

Tested the process deployment

Congratulations! You successfully deployed all three processes and tested the connection to your main cluster.

Understand business requirement

Your first task is to connect the **Business Travel** process with the **Travel Agency** process. Fortunately, BPMN offers a variety of interprocess communication methods to resolve this, including **message events**, **call activities**, and **signals**.

Analysis

Learn more about each of these communication methods below:

Call activities are good for readability, maintainability, and reusability.

Considerations:

- Both process must be deployed in the same cluster (this is known as a tightly coupled cluster). To launch a new instance, a call activity utilizes the name of the called process. The parent process must be launched in the same cluster as the called process.
- All process variables are shared from the parent process to the called process. You may not want to share all your business data to a third party.
- You could improve your Business Travel process with a call activity to the Travel Agency process, which will handle the booking and return the confirmation. *However, this process is deployed in the Travel Agency cluster, and you don't want to share any private business data from your parent process*.

Signal events are good for broadcast communication.

Considerations:

- A single signal event can communicate with multiple signals that are listening when the signal is broadcast. This can be useful when starting an instance of multiple processes at the same time, or for alerting multiple processes at once.
- You could improve the Business Travel process with a signal event to start the Travel Agency process. While this technically works, *signal events are primarily used for broadcasting communication from 1 to n processes*.

Message events are good for asynchronous interprocess communication.

Considerations:

- Message events use correlation to communicate end-to-end with a process instance. This can be useful when starting an instance of a different process, sending some data to another process and continuing the process flow, stopping the process flow and waiting for data sent by another process, synchronizing two process instances, and more.
- You could improve the Business Travel process by sending a message to start the Travel Agency process. You could also wait until you receive the booking confirmation using a message event. Message events are cross-cluster: you can send a message to a process in another cluster.

Conclusion

In this scenario, the Travel Agency process is provided as if the travel agency is using its own cluster. For this use case, the communication method you have chosen is for your company to **send a message** to start the booking process once the consultant has selected their flight and

hotel, and then wait to receive a booking confirmation message. That way, the consultant will not have to call the travel agency each time they need to plan a business trip.

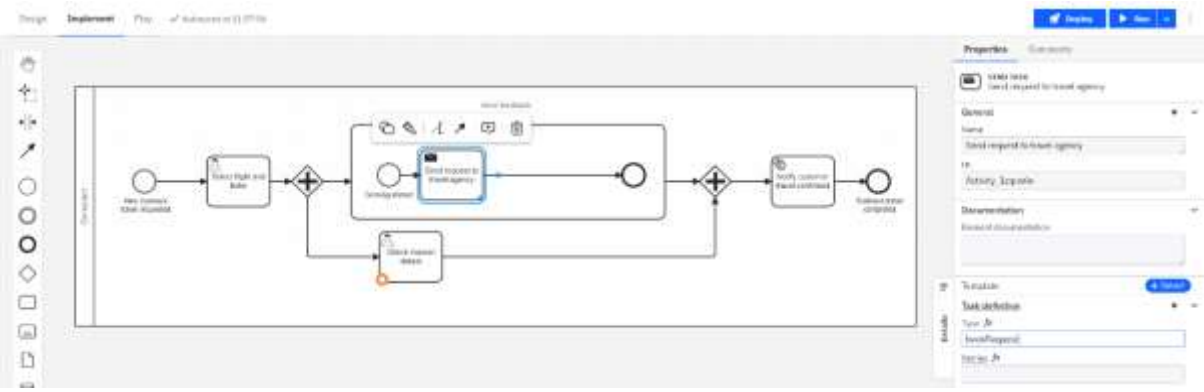
In the next lesson, you will begin this implementation. You will start by focusing on automating and delegating the **Booking** subprocess.

Implement adjustments

Implement the Business Travel process

Add a send task

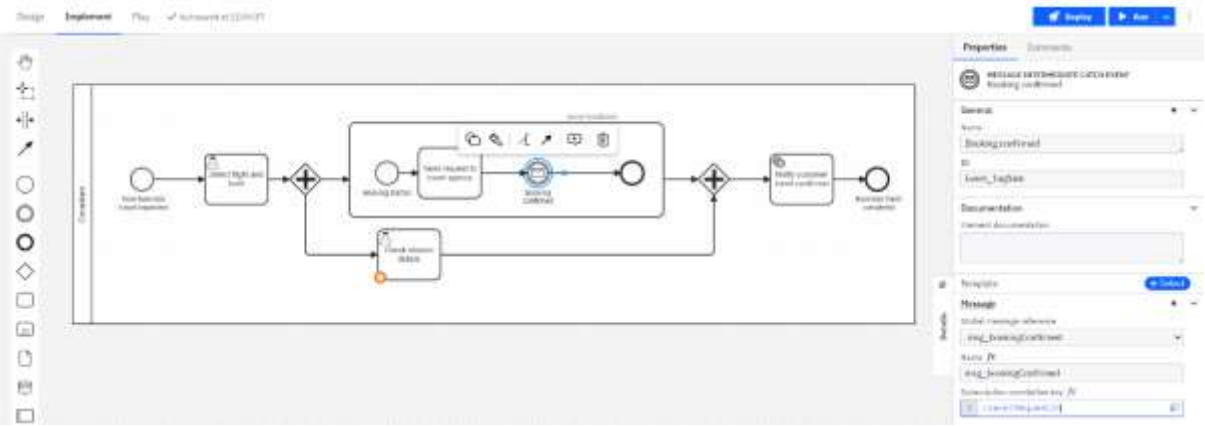
1. In Camunda 8 Web Modeler, navigate to the **Business Travel** process.
2. In the **Booking** subprocess, after the **Booking started** start event, add a **send task** and name it "Send request to travel agency".
3. Update the task properties:
    - Navigate to **Properties** > **Task definition**.
    - In the **Type** field, enter "bookRequest".



Send task added to Business Travel process

Add a message intermediate catch event

1. After the send task, add a **message intermediate catch event**, and name it "Booking confirmed".
2. Update the message properties:
    - Navigate to **Properties** > **Message** > **Global message reference** and select **Create new...**.
    - In the **Name** field, enter "msg_bookingConfirmed".
    - In the **Subscription correlation key** field, enter "travelRequestId".



Catch event added to Business Travel process

Deploy the Business Travel process

1. Deploy the updated Business Travel process (click **Deploy** > **Deploy**).

Implement the Booking handler

1. In your IDE, navigate to the **BusinessTravelApplication** class.
2. Create a constant variable for the job type:

```
private static final String BOOK_REQUEST_JOB_TYPE = "bookRequest";
```

3. Create a new job worker for the job type:

```
final JobWorker bookingRequestWorker =
      client.newWorker()
          .jobType(BOOK_REQUEST_JOB_TYPE)
          .handler(new BookingRequestHandler())
          .timeout(Duration.ofSeconds(WORKER_TIMEOUT).toMillis())
          .open();
```

4. Create the **BookingRequestHandler class** in `src/main/java/com/camunda/academy/handler`. This will implement **JobHandler**.
5. Create a constant variable for the message name:

```
private static final String MESSAGE_NAME = "msg_startBookingRequest";
```

Message name

You can check the start message name in the Travel Agency process.

6. Get the process variables into the `handle()` method:

```
final Map<String, Object> inputVariables = job.getVariablesAsMap();
final String travelRequestId = (String) inputVariables.get("travelRequestId");
```

```
final String travelDestination = (String)
inputVariables.get("travelDestination");
final String travelDate = (String) inputVariables.get("travelDate");
final String travelFlight =  (String) inputVariables.get("travelFlight");
final String travelHotel =  (String) inputVariables.get("travelHotel");
```

Info

You are handling a job worker from your cluster. You should connect to the Zeebe engine of the cluster where the message will be caught, so you can send the message there.

7. Create a new client for the cluster where the Travel Agency process is deployed. This method will load your cluster credentials file:

```
public static Properties loadProperties(String fileName) throws IOException {
    InputStream input = new FileInputStream(fileName);
        Properties prop = new Properties();
        prop.load(input);
    return prop;
}
```

Now, create the new client connection for the cluster where the Travel Agency process is deployed.

```
Properties properties =
loadProperties("src/main/resources/travelagency.cluster.properties");

final String ZEEBE_ADDRESS = properties.getProperty("ZEEBE_ADDRESS");
final String ZEEBE_CLIENT_ID = properties.getProperty("ZEEBE_CLIENT_ID");
final String ZEEBE_CLIENT_SECRET =
properties.getProperty("ZEEBE_CLIENT_SECRET");
final String ZEEBE_AUTHORIZATION_SERVER_URL =
properties.getProperty("ZEEBE_AUTHORIZATION_SERVER_URL");
final String ZEEBE_TOKEN_AUDIENCE =
properties.getProperty("ZEEBE_TOKEN_AUDIENCE");


final OAuthCredentialsProvider credentialsProvider =
    new OAuthCredentialsProviderBuilder()
      .authorizationServerUrl(ZEEBE_AUTHORIZATION_SERVER_URL)
        .audience(ZEEBE_TOKEN_AUDIENCE)
        .clientId(ZEEBE_CLIENT_ID)
        .clientSecret(ZEEBE_CLIENT_SECRET)
        .build();

try (final ZeebeClient travelAgencyClient = ZeebeClient.newClientBuilder()
          .gatewayAddress(ZEEBE_ADDRESS)
          .credentialsProvider(credentialsProvider)
          .build()) {

}
```

8. Map the process variables you will send in your message:

```
final Map<String, Object> messageVariables = new HashMap<String, Object>();

messageVariables.put("travelRequestId", travelRequestId);
messageVariables.put("travelDestination", travelDestination);
messageVariables.put("travelDate", travelDate);
messageVariables.put("travelFlight", travelFlight);
messageVariables.put("travelHotel", travelHotel);
```

Correlation key

One important feature of messages is the correlation key, which uniquely identifies the receiver of the message. Your correlation key is `travelRequestId`.

9. Send the message using the PublishMessage RPC Command. You will need to pass the message name, the correlation key, and the message variables.

```
travelAgencyClient.newPublishMessageCommand()
      .messageName(MESSAGE_NAME)
      .correlationKey(travelRequestId)
      .variables(messageVariables)
      .send()
      .join();
```

10. Add a console message to indicate that the travel request was started.

```
System.out.println(travelRequestId + " Travel Request started");
```

Complete the job

You sent the message using the Travel Agency cluster client. Now you have to complete the job of the send task in the Business Travel process. You will use the main cluster client.

11. Send the complete job command:

```
client.newCompleteCommand(job.getKey()).send().join();
```

12. Once you have made the relevant changes, the final version of the **handle(JobClient client, ActivatedJob job)** method should look similar to the following:

```
@Override
  public void handle(JobClient client, ActivatedJob job) throws Exception {

    //Obtain the Process Variables
    final Map<String, Object> inputVariables = job.getVariablesAsMap();
    final String travelRequestId = (String)
inputVariables.get("travelRequestId");
```

```
    final String travelDestination = (String)
inputVariables.get("travelDestination");
    final String travelDate = (String) inputVariables.get("travelDate");
    final String travelFlight =  (String) inputVariables.get("travelFlight");
    final String travelHotel =  (String) inputVariables.get("travelHotel");

    //Load credentials from resources/config.properties file
    Properties properties =
loadProperties("resources/travelagency.cluster.properties");

    final String ZEEBE_ADDRESS = properties.getProperty("ZEEBE_ADDRESS");
    final String ZEEBE_CLIENT_ID = properties.getProperty("ZEEBE_CLIENT_ID");
    final String ZEEBE_CLIENT_SECRET =
properties.getProperty("ZEEBE_CLIENT_SECRET");
    final String ZEEBE_AUTHORIZATION_SERVER_URL =
properties.getProperty("ZEEBE_AUTHORIZATION_SERVER_URL");
    final String ZEEBE_TOKEN_AUDIENCE =
properties.getProperty("ZEEBE_TOKEN_AUDIENCE");


    final OAuthCredentialsProvider credentialsProvider =
        new OAuthCredentialsProviderBuilder()
          .authorizationServerUrl(ZEEBE_AUTHORIZATION_SERVER_URL)
            .audience(ZEEBE_TOKEN_AUDIENCE)
            .clientId(ZEEBE_CLIENT_ID)
            .clientSecret(ZEEBE_CLIENT_SECRET)
            .build();

    try (final ZeebeClient travelAgencyClient = ZeebeClient.newClientBuilder()
              .gatewayAddress(ZEEBE_ADDRESS)
              .credentialsProvider(credentialsProvider)
              .build()) {

        //Build the Message Variables
      final Map<String, Object> messageVariables = new HashMap<String,
Object>();

      messageVariables.put("travelRequestId", travelRequestId);
      messageVariables.put("travelDestination", travelDestination);
      messageVariables.put("travelDate", travelDate);
      messageVariables.put("travelFlight", travelFlight);
      messageVariables.put("travelHotel", travelHotel);

      //Send the message
      travelAgencyClient.newPublishMessageCommand()
        .messageName(MESSAGE_NAME)
        .correlationKey(travelRequestId)
        .variables(messageVariables)
        .send()
        .join();

      System.out.println(travelRequestId + " Travel Request started");

      //Complete the Job
```

```
client.newCompleteCommand(job.getKey()).variables(messageVariables).send().join();
    }
}

public static Properties loadProperties(String fileName) throws IOException {
    InputStream input = new FileInputStream(fileName);
      Properties prop = new Properties();
      prop.load(input);
  return prop;
  }
```

Process improved!

You improved the Business Travel process by using a send task to start an instance of the Travel Agency process, and a catch message event to get the booking confirmation from the Travel Agency process.

Application logic improved!

You improved the logic of the Business Travel application by using a handler of the send task job worker that connects to another cluster to send a message to start a new instance of the Travel Agency process.

Run & test process

Deploy process

1. In Camunda 8 Web Modeler, navigate to the **Business Travel** process and then deploy it to your main cluster.

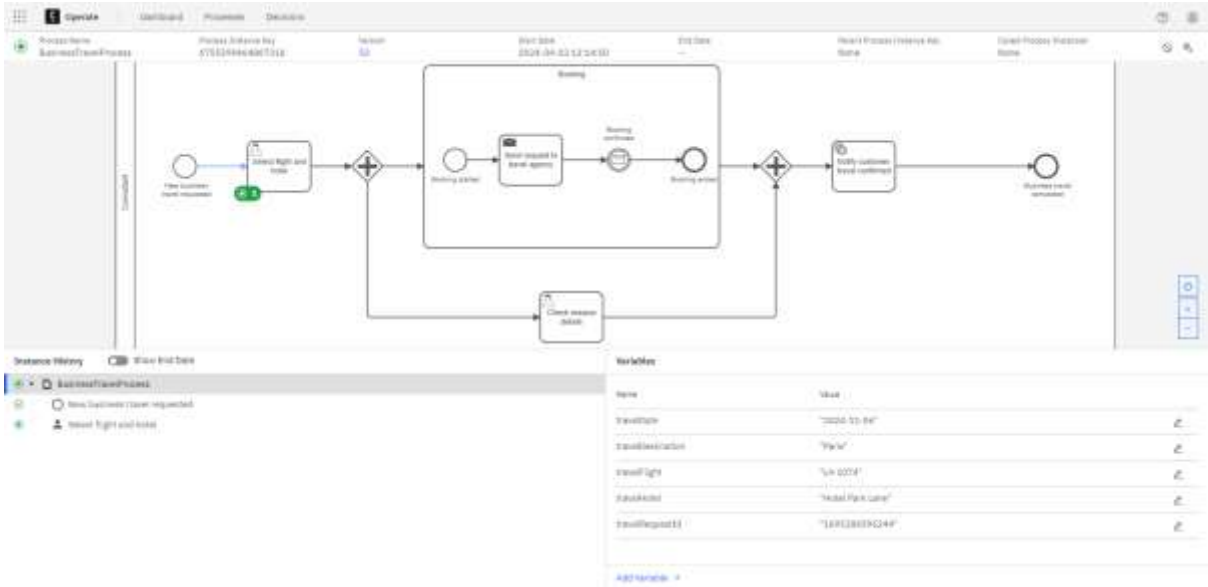
Run and test

1. In your IDE, clear all previous launches and then run both the `BusinessTravelApplication.java` and the `TravelAgencyApplication.java` classes.
2. From the Camunda 8 **Modeler**, run a new Business Travel process instance using the following JSON as an input:

```
{
  "travelDate": "2024-11-06",
  "travelDestination": "Paris",
  "travelFlight": "LH 1074",
  "travelHotel": "Hotel Park Lane",
  "travelRequestId": "1694681171118"
}
```

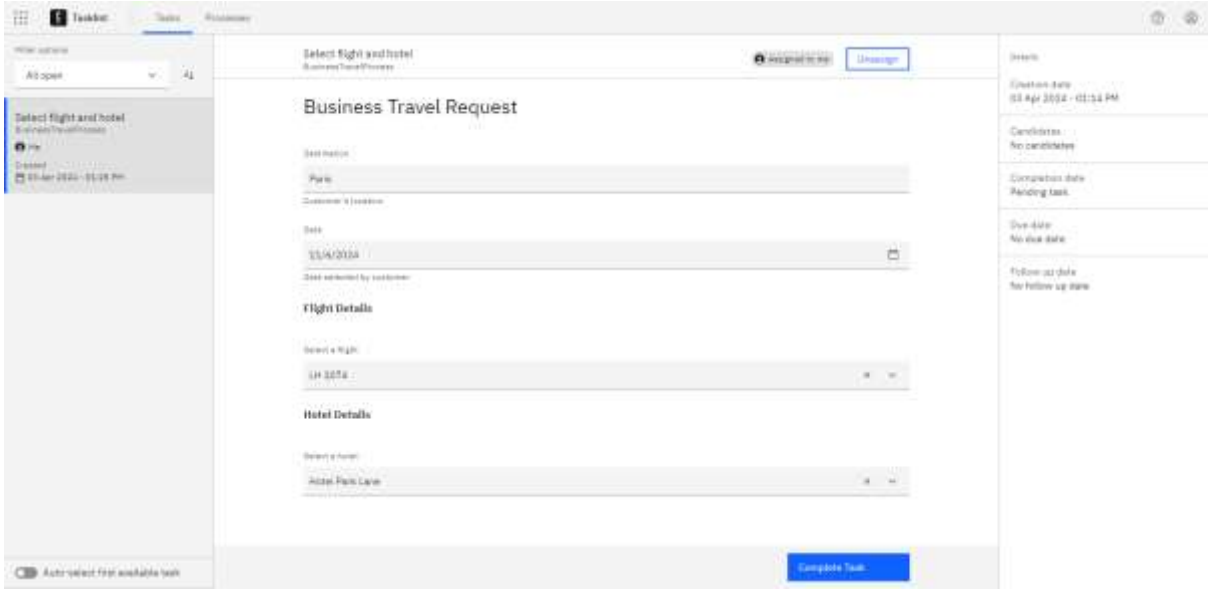
3. Navigate to Camunda 8 **Operate**.
4. Confirm that there is a running instance of the **BusinessTravelProcess**.

5. When the instance arrives at the **Select flight and hotel** user task, proceed to the next step.



Select flight and hotel user task

6. Navigate to Camunda 8 **Tasklist**.
7. Assign the **Select flight and hotel** user task to yourself and complete it.



Business Travel Request form

8. Navigate to the IDE console to review the last printed messages from the `Business Travel` application:

```
Connected to Cluster 1: TopologyImpl{brokers=[BrokerInfoImpl{nodeId=1,
host='zeebe-1.zeebe-broker-service.xxx-zeebe.svc.cluster.local', port=26501,
version=8.5.0, partitions=[PartitionInfoImpl{partitionId=1, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=2, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=3, role=LEADER,
```

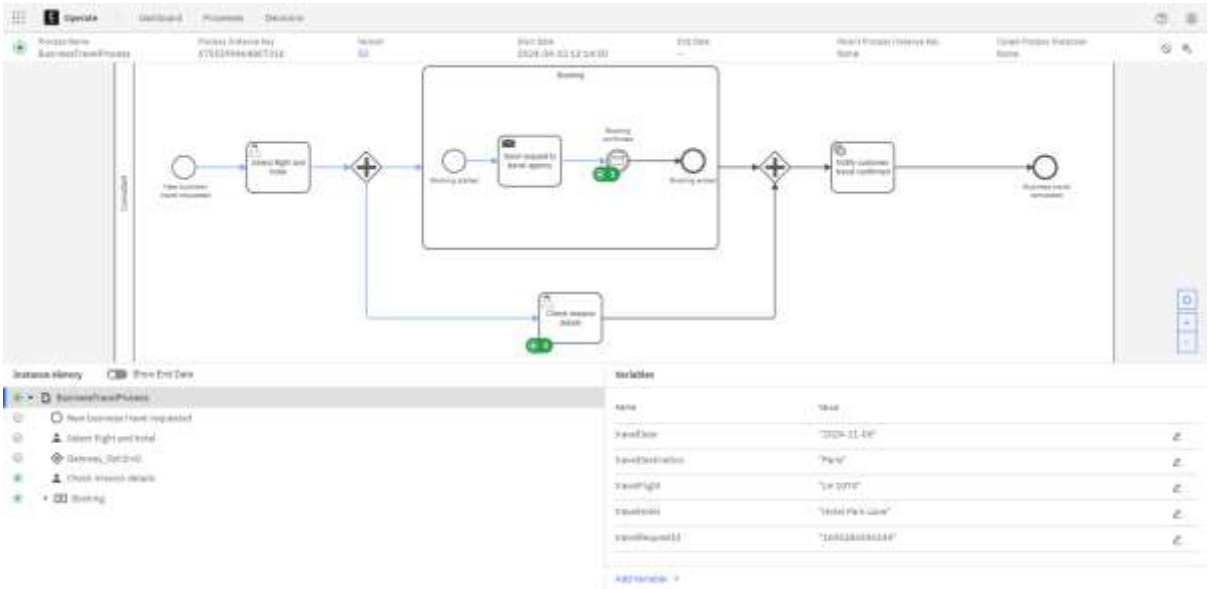
```
health=HEALTHY}]}, BrokerInfoImpl{nodeId=0, host='zeebe-0.zeebe-broker-
service.xxx-zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]},
BrokerInfoImpl{nodeId=2, host='zeebe-2.zeebe-broker-service.xxx-
zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]}],
clusterSize=3, partitionsCount=3, replicationFactor=3, gatewayVersion=8.5.0}
1694681171118 Travel Request started
```

Travel request started!

There's a new printed message proving that the Publish Message command worked! A new Travel Agency process instance was created.

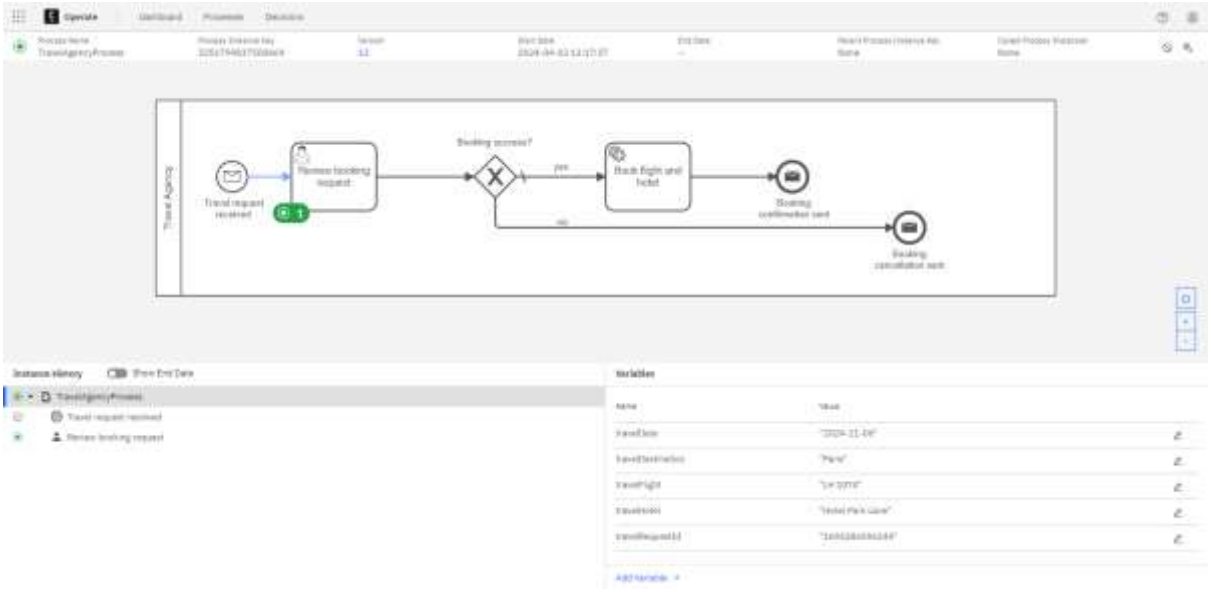
9. Navigate to Camunda 8 **Operate**.
10. Verify that there are now two instances:

    - The **BusinessTravelProcess** instance is waiting for the **Booking confirmed** message event and the **Check mission details** user task.



Business Travel process waiting message

    - The **TravelAgencyProcess** instance is on the **Review booking request** user task.

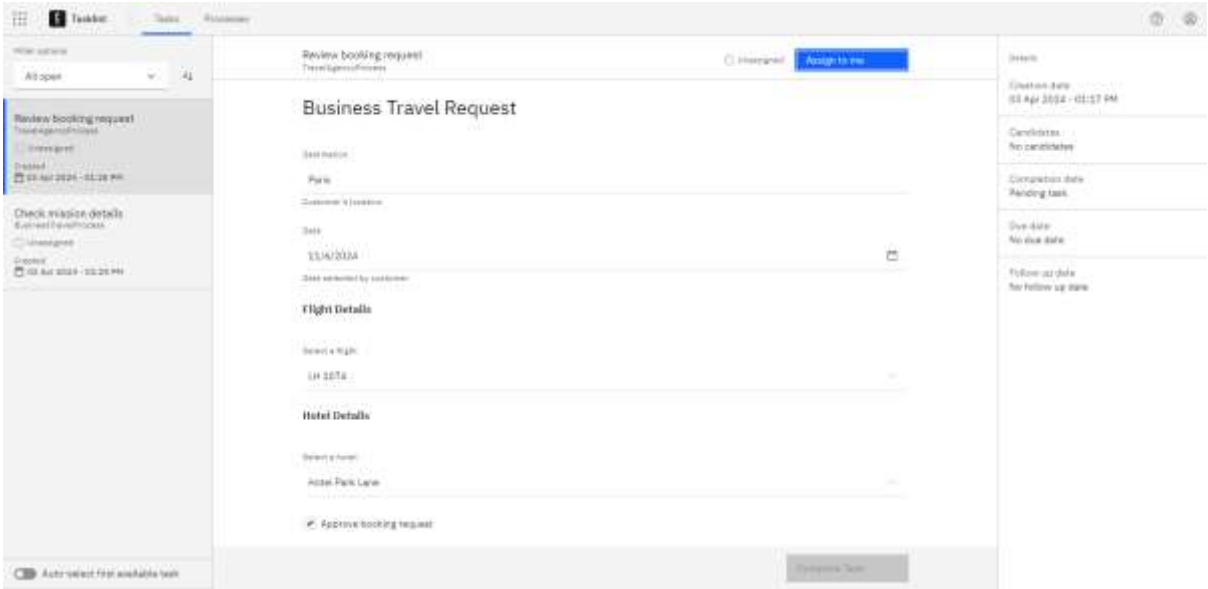


Travel Agency process user task

Multiple clusters

If you are following this course using the multiple clusters approach, you will have to verify that the **TravelAgencyProcess** instance is in the second cluster, and perform the user tasks there as well.

11. Navigate to Camunda 8 **Tasklist**.
12. Assign the **Review booking request** user task to yourself.
13. Verify that "Approve booking request" is selected, then complete the task.



Review booking request user task form

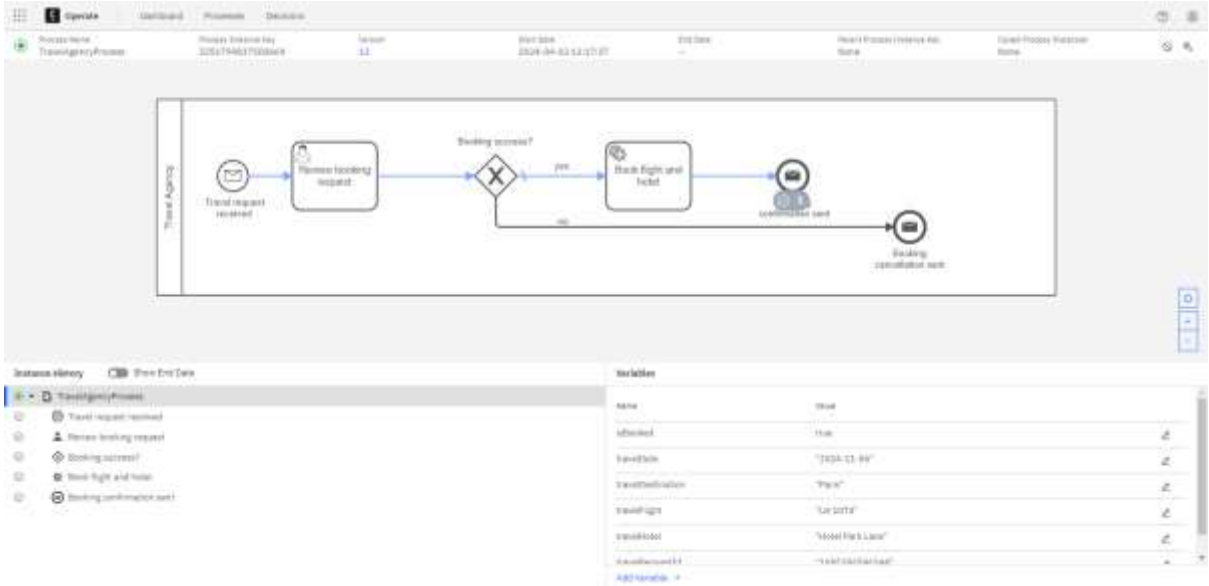
14. Navigate to the IDE console to review the last printed messages from the `Travel Agency` application:

```
Connected to Cluster 2: TopologyImpl{brokers=[BrokerInfoImpl{nodeId=1,
host='zeebe-1.zeebe-broker-service.xxx-zeebe.svc.cluster.local', port=26501,
version=8.5.0, partitions=[PartitionInfoImpl{partitionId=1, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=2, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=3, role=LEADER,
health=HEALTHY}]}, BrokerInfoImpl{nodeId=0, host='zeebe-0.zeebe-broker-
service.xxx-zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]},
BrokerInfoImpl{nodeId=2, host='zeebe-2.zeebe-broker-service.xxx-
zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]}],
clusterSize=3, partitionsCount=3, replicationFactor=3, gatewayVersion=8.5.0}
1694681171118 Travel Request: Flight and Hotel booked
1694681171118 Travel Request: Confirmation sent
```

Info

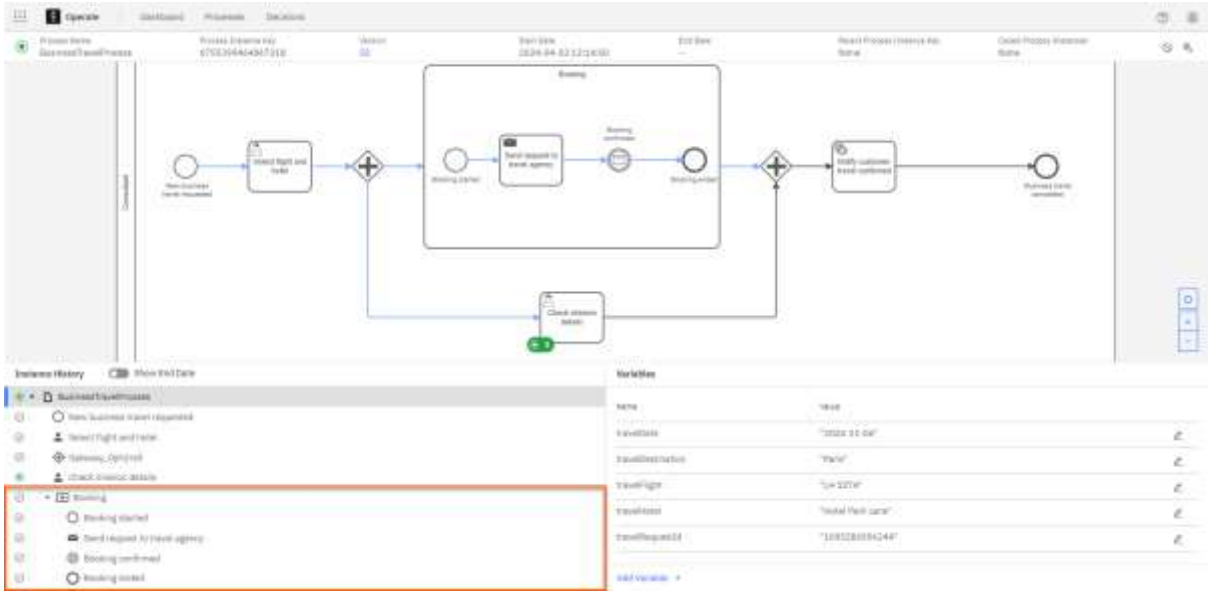
After running the Travel Agency application all of the pending service tasks and send tasks were automatically handled.

15. Navigate to Camunda 8 **Operate** and verify that the **TravelAgencyProcess** instance is now completed.



Travel Agency process instance completed

16. Now, check the Business Travel process instance to verify that the booking was completed.



Business Travel process booking completed

Booking confirmed!

The booking is completed and confirmed. Next, check the mission details and get prepared for the business trip. After that, your customer will receive the business travel confirmation.

17. From Camunda 8 **Tasklist**, assign the **Check mission details** user task to yourself and complete it.
18. Return to the IDE console to review the last printed messages from the `Business Travel` application:

```
Connected to Cluster 1: TopologyImpl{brokers=[BrokerInfoImpl{nodeId=1,
host='zeebe-1.zeebe-broker-service.xxx-zeebe.svc.cluster.local', port=26501,
version=8.5.0, partitions=[PartitionInfoImpl{partitionId=1, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=2, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=3, role=LEADER,
health=HEALTHY}]}, BrokerInfoImpl{nodeId=0, host='zeebe-0.zeebe-broker-
service.xxx-zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]},
BrokerInfoImpl{nodeId=2, host='zeebe-2.zeebe-broker-service.xxx-
zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]}],
clusterSize=3, partitionsCount=3, replicationFactor=3, gatewayVersion=8.5.0}
1694681171118 Travel Request started
1694681171118 Business Travel confirmed
```

Send a message to the Travel Agency process

Congratulations! You successfully sent a message to start a new booking request.

Catch a confirmation message from the Travel Agency process

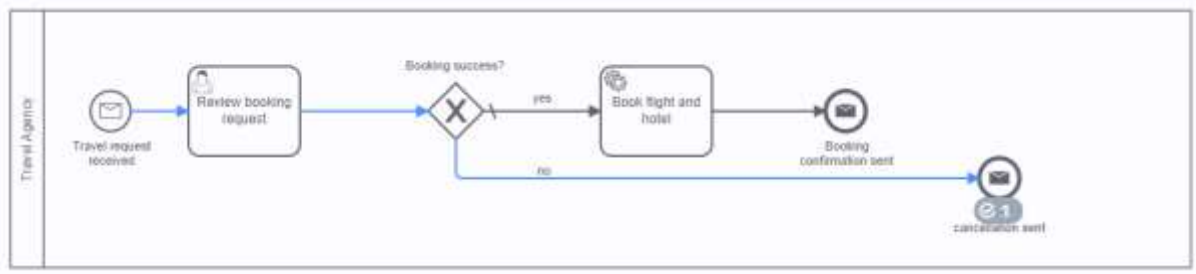
Congratulations! You successfully caught a message to confirm the booking was completed.

Understand business requirement

So far, you've been able to send a message to start the booking from the Travel Agency process and catch a message once the booking is confirmed.

Your next task is to deal with a possible scenario in which something goes wrong in the **Travel Agency** process, and the booking request has to be cancelled.

In the simplest version of the Travel Agency process, the end user employee deselects "Approve booking request" in the **Review booking request** user task form. Then, the instance takes the cancellation path in the process and the cancellation message is sent.



Travel Agency process cancellation path

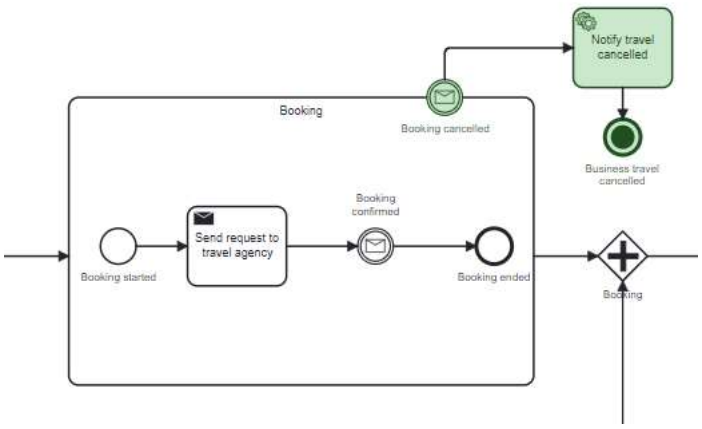
Analysis

Review the following options to learn how you can catch this cancellation message event in your Business Travel process.

Message boundary event

One option is to add a **message boundary event** to the boundary of the **Booking** expanded subprocess.

This strategy works well with expanded subprocesses, since they accept intermediate boundary events. In this case, the process instance will wait until the booking is confirmed in the catch message event. If something goes wrong, the expanded subprocess will catch the boundary message event, cancel the catch message event, and continue the flow to the **Notify travel cancelled** task, and then to the **Business travel cancelled** end event.



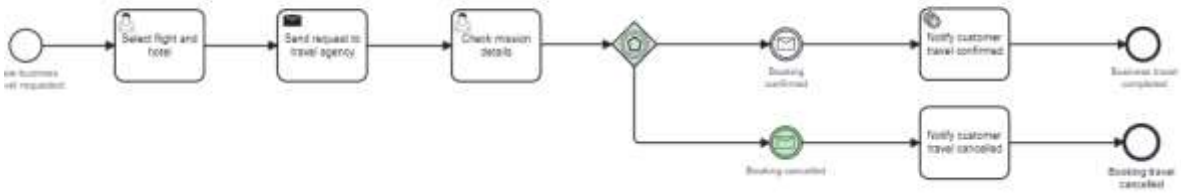
Business Travel process cancellation with a message boundary event

Info

This option will end the process instance, no matter which BPMN element is active.

Event-based gateway

Another commonly used approach for catching events is to use an **event-based gateway**. The first caught event will trigger the path flow. In this case, if the booking confirmation message is sent, the process flow will continue to the **Notify customer travel confirmed** task, and then to the end event. Otherwise, if the booking cancellation message is sent, the process flow will continue to the **Notify customer travel cancelled** task, and then the end event.



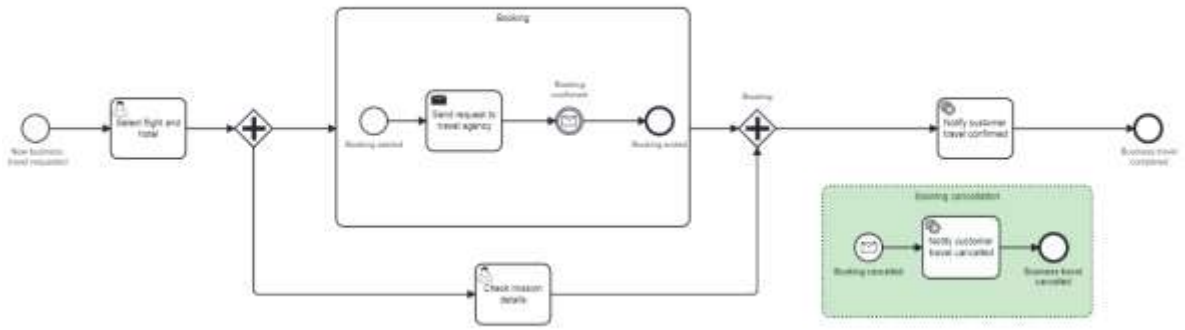
Business Travel process cancellation with an event-based gateway

Event subprocess

Another option is to use an **event subprocess**. This type of special subprocess can only be reached by certain events, including:

- **Timer start** events
- **Error start** events
- **Escalation start** events
- **Signal start** events
- **Conditional start** events
- **Compensation start** events
- **Message start** events

In this case, if the booking cancellation message is sent, the process instance will cancel any active task, and the process flow will continue to the **Notify customer travel cancelled** task, and then to the end event.



Business Travel process cancellation with an event subprocess

Info

This will end the process instance, no matter which BPMN element is active.

Conclusion

There are a variety of cancellation strategies that could be implemented in your Business Travel process. One consideration is that if the process evolves in the future, it may be wise to cancel the entire process instance. For that reason, you will use the **event subprocess** approach for the next process implementation.

Tip

Keep in mind that all of these strategies are valid approaches for handling process instance cancellations.

Implement adjustments

Implement the Business Travel process

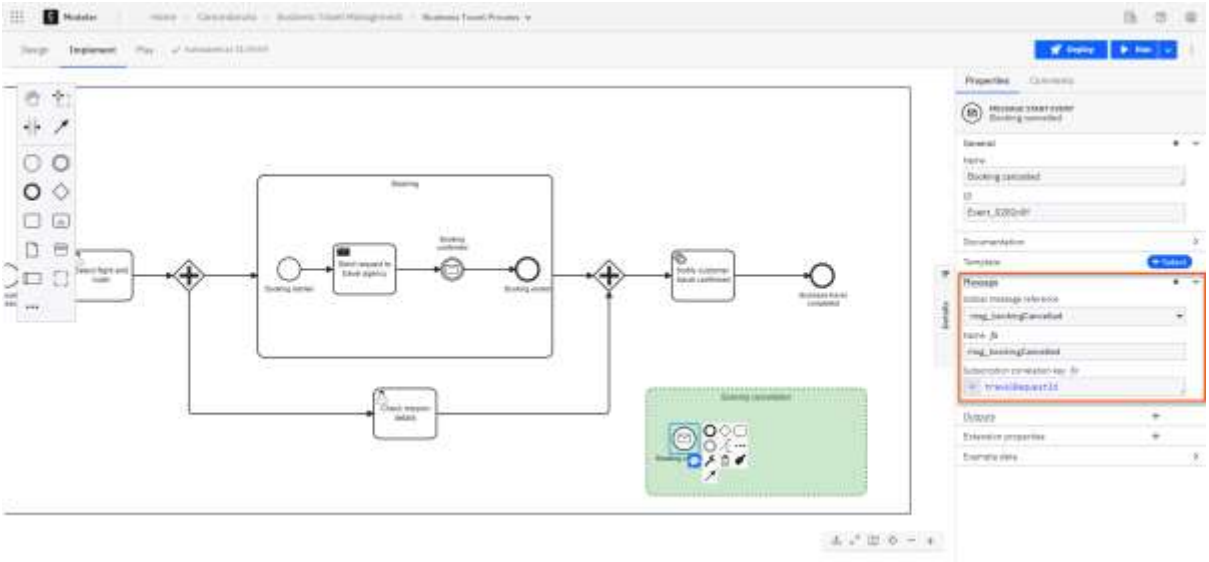
Add an event subprocess

1. In Camunda 8 Web Modeler, navigate to the **Business Travel** process.
2. At the bottom right corner of the process, add an **event subprocess** and name it "Booking cancellation".

Add a message start event

1. In the **Booking cancellation** event subprocess, add a **message start** event and name it "Booking cancelled".
2. Update the message properties:
    - Navigate to **Properties** > **Message** > **Global message reference** and select **Create new...**.
    - In the **Name** field, enter "msg_bookingCancelled".
    - In the **Subscription correlation key** field, enter "travelRequestId".

The process will now look like this:



Business Travel process with cancellation message set up

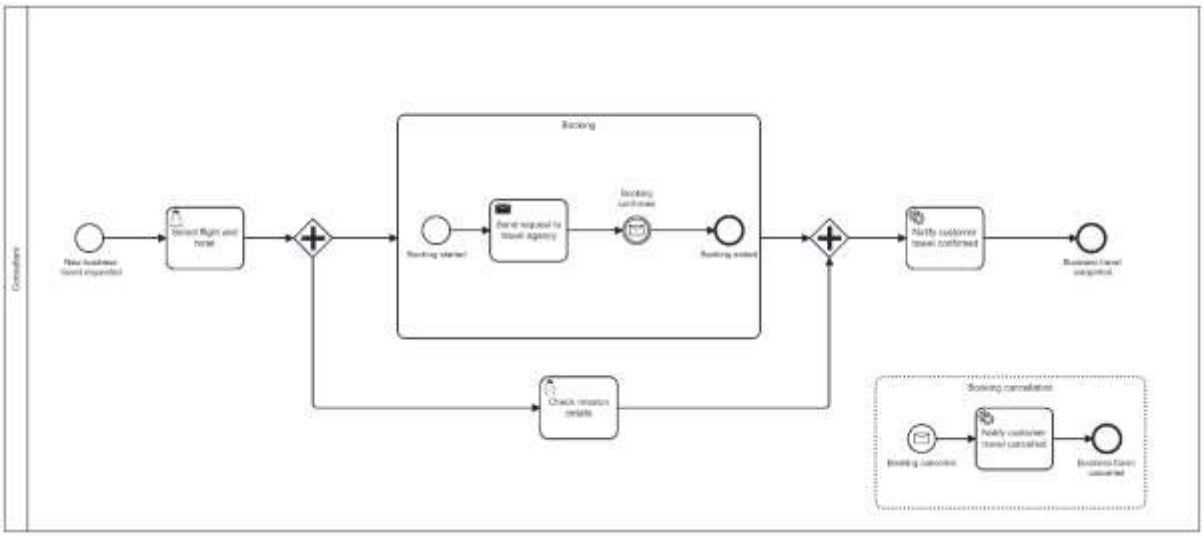
Add a service task

1. After the message event, add a **service task** and name it "Notify customer travel cancelled".
2. Update the task properties:
    - Navigate to **Properties** > **Task definition**.
    - In the **Type** field, enter "notifyCustomerTravelCancelled".

Add an end event

1. Finish your event subprocess by adding an **end event** and naming it "Business travel cancelled".

The process will now look like this:



Business Travel process with booking cancellation event subprocess completed

Deploy the Business Travel process

1. Deploy the updated **Business Travel** process.

Implement the Notify Customer Travel Cancelled handler

1. In your IDE, navigate to the **BusinessTravelApplication** class.
2. Create a constant variable for the job type:

```
private static final String NOTIFY_CUSTOMER_TRAVEL_CANCELLED_JOB_TYPE
="notifyCustomerTravelCancelled";
```

3. Create a new worker for the job type:

```
final JobWorker notifyCustomerBookingCancelledWorker =
      client.newWorker()
          .jobType(NOTIFY_CUSTOMER_TRAVEL_CANCELLED_JOB_TYPE)
          .handler(new NotifyCustomerTravelCancelledHandler())
          .timeout(Duration.ofSeconds(WORKER_TIMEOUT).toMillis())
          .open();
```

4. Create the **NotifyCustomerTravelCancelledHandler** class in `src/main/java/com/camunda/academy/handler`. This implements **JobHandler**.
5. Create an instance of the **CustomerService** class:

```
CustomerService customerService = new CustomerService();
```

6. Get the process variables into the `handle()` method:

```
final Map<String, Object> inputVariables = job.getVariablesAsMap();
final String travelRequestId = (String) inputVariables.get("travelRequestId");
```

7. Use the method `notifyTravelCancelled()` from the **CustomerService** class:

```
customerService.notifyTravelCancelled(travelRequestId);
}
```

8. Send the complete job command:

```
client.newCompleteCommand(job.getKey()).send().join();
```

9. After making the relevant changes, the final version of the **handle(JobClient client, ActivatedJob job)** method should look similar to the following:

```
@Override
public void handle(JobClient client, ActivatedJob job) throws Exception {

      final Map<String, Object> inputVariables = job.getVariablesAsMap();
      final String travelRequestId = (String)
inputVariables.get("travelRequestId");

      //Notify customer
      customerService.notifyTravelCancelled(travelRequestId);

      //Complete the Job
      client.newCompleteCommand(job.getKey()).send().join();
}
```

Process improved!

You improved the Business Travel process by using an event subprocess, which catches the cancellation message and then sends a notification before finishing the whole process instance.

Application logic improved!

You improved the logic of the Business Travel application with a handler of the service task, which sends a notification if the business travel is cancelled.

Run & test process

Deploy the process

1. In Camunda 8 Web Modeler, navigate to the **Business Travel** process and deploy it to your main cluster.

Run and test

1. Repeat the initial steps from the **Process communication - Run & test** lesson:
    - Run both the `BusinessTravelApplication.java` and `TravelAgencyApplication.java` classes.
    - Run a new Business Travel process instance from the Camunda 8 **Modeler** using the following JSON as an input:

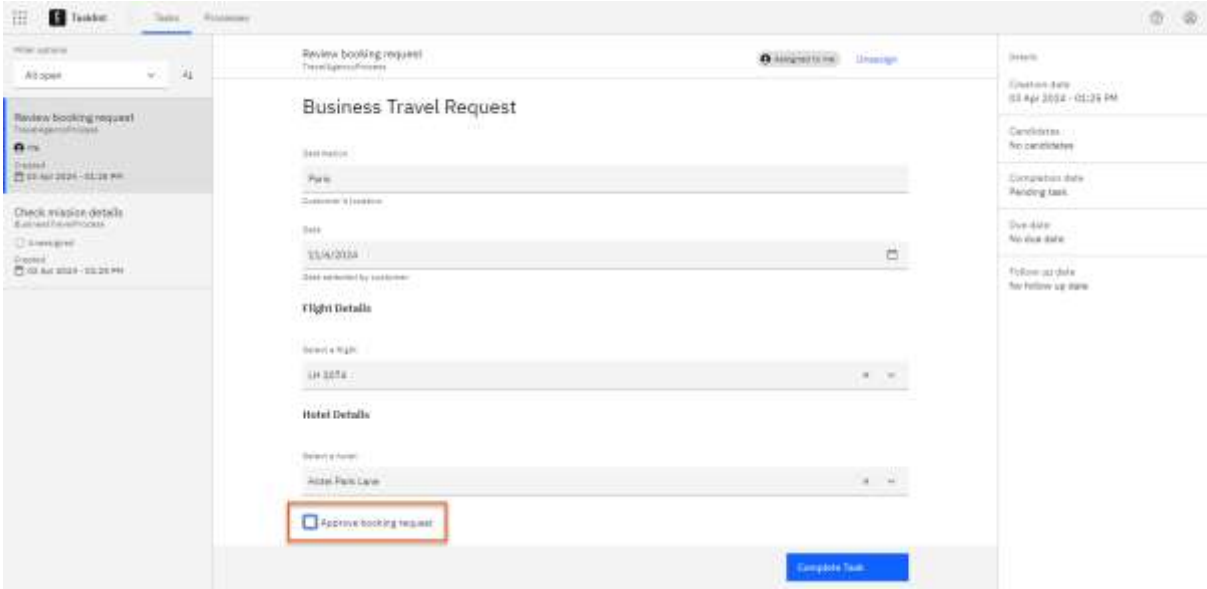
```
{
  "travelDate": "2024-11-06",
  "travelDestination": "Paris",
  "travelFlight": "LH 1074",
  "travelHotel": "Hotel Park Lane",
  "travelRequestId": "1694773066287"
}
```

    - ...

Booking request not approved

You are testing the cancellation path of the process so you won't approve the booking request this time.

1. Assign the **Review booking request** user task to yourself.
2. **Deselect** "Approve booking request" and then complete the task.



Booking request not approved

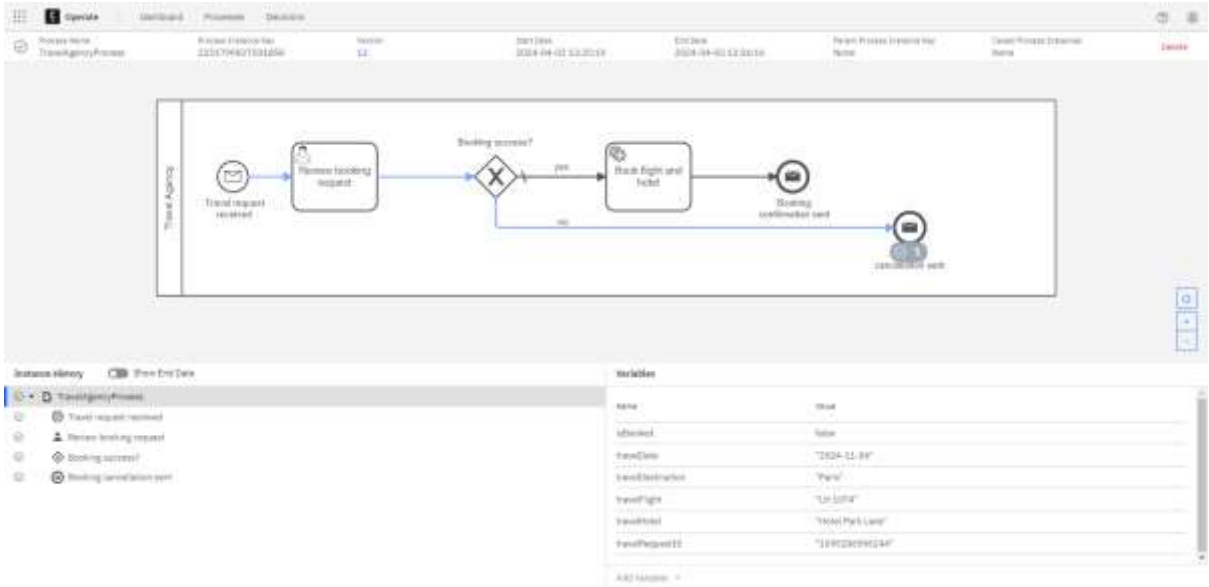
3. Navigate to the IDE console to review the last printed messages from the `Travel Agency` application:

```
Connected to Cluster 2: TopologyImpl{brokers=[BrokerInfoImpl{nodeId=1,
host='zeebe-1.zeebe-broker-service.xxx-zeebe.svc.cluster.local', port=26501,
version=8.5.0, partitions=[PartitionInfoImpl{partitionId=1, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=2, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=3, role=LEADER,
health=HEALTHY}]}, BrokerInfoImpl{nodeId=0, host='zeebe-0.zeebe-broker-
service.xxx-zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]},
BrokerInfoImpl{nodeId=2, host='zeebe-2.zeebe-broker-service.xxx-
zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]}],
clusterSize=3, partitionsCount=3, replicationFactor=3, gatewayVersion=8.5.0}
1694773066287 Travel Request: Booking cancellation sent
```

Booking request cancelled

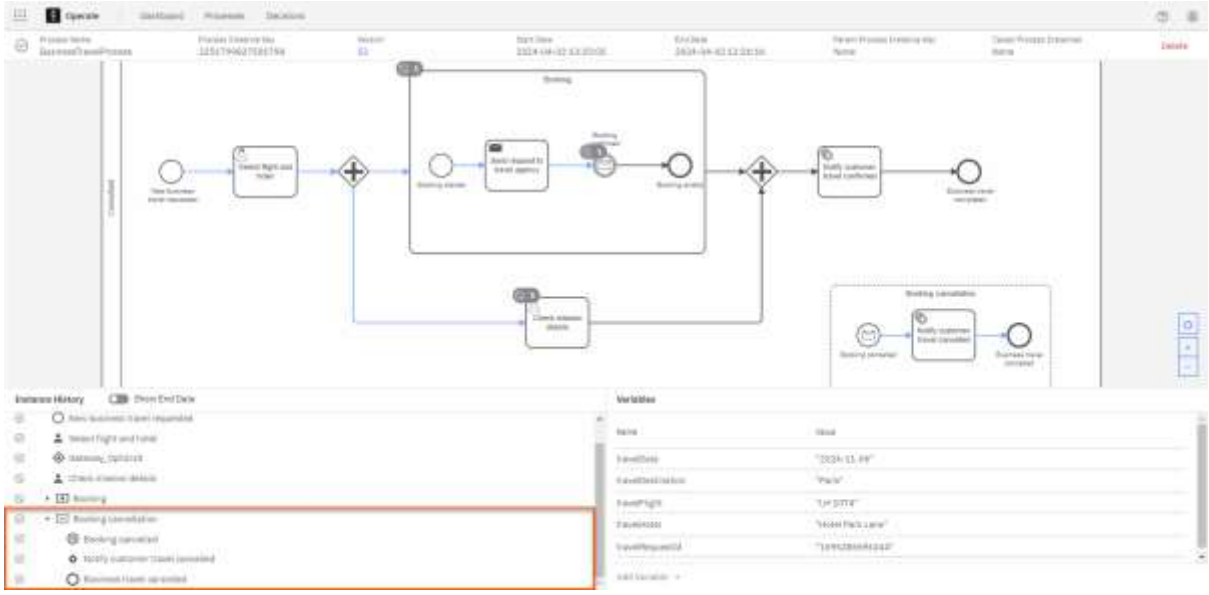
There's a new printed message announcing that the booking cancellation message was sent.

4. Return to Camunda 8 **Operate** and verify that the **TravelAgencyProcess** instance is now completed under the cancellation path.



Travel Agency process with cancellation path completed

5. Check the **Business Travel** process instance to verify the booking was cancelled.



Business Travel process with booking cancelled

6. Return to the IDE console to review the last printed messages from the `Business Travel` application:

```
Connected to Cluster 1: TopologyImpl{brokers=[BrokerInfoImpl{nodeId=1,
host='zeebe-1.zeebe-broker-service.xxx-zeebe.svc.cluster.local', port=26501,
version=8.5.0, partitions=[PartitionInfoImpl{partitionId=1, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=2, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=3, role=LEADER,
health=HEALTHY}]}, BrokerInfoImpl{nodeId=0, host='zeebe-
0.dealingWithCancellation-implementAdjustments copyzeebe-broker-service.xxx-
zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
```

```
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]},
BrokerInfoImpl{nodeId=2, host='zeebe-2.zeebe-broker-service.xxx-
zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]}],
clusterSize=3, partitionsCount=3, replicationFactor=3, gatewayVersion=8.5.0}
1694773066287 Travel Request started
1694773066287 Bussiness Travel cancelled
```

Business Travel cancelled!

Now the Business Travel process is cancelled.

Catch a cancellation message from the Travel Agency process

Congratulations! You successfully caught a message to confirm the booking was cancelled.

Call activity communication

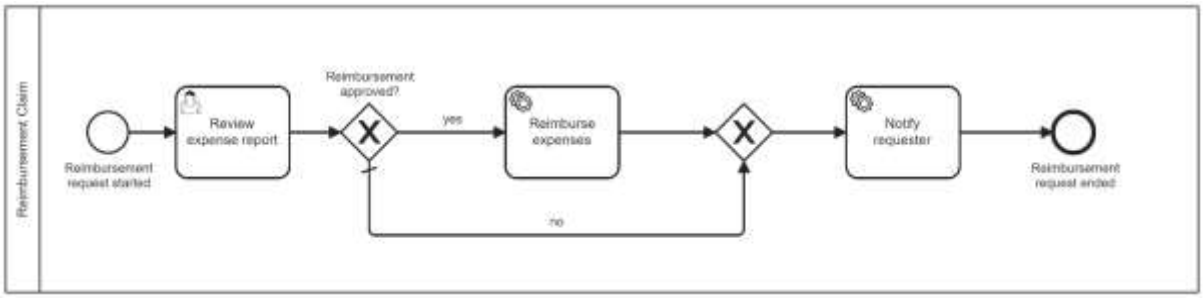
Understand business requirement

Next, the consultant must provide input about the business trip and specify whether any expenses should be reimbursed.

Analysis

Reimbursement Claim process

The **Reimbursement Claim** process is widely used across departments in your company each time an employee needs to reimburse an expense.



Reimbursement Claim process

This process handles the reimbursement requests:

- An employee reviews the expense report (**Review expense report**).
- If it's approved, the expenses are reimbursed (**Reimburse expenses**). If it's not approved, then this task is skipped.
- The employee is then notified about the decision (**Notify requester**), and the process ends (**Reimbursement request ended**).

Conclusion

You can improve the Business Travel process by adding a **call activity** to the **Reimbursement Claim** process to handle cases where expense reimbursements need to be claimed. In cases where there's no need for expense reimbursements, the process will be completed.

Reminder

Call activities are good for readability, maintainability, and reusability.

In the next lesson you will implement these changes.

Implement adjustments

Implement the Business Travel process

Add a user task

1. In Camunda 8 Web Modeler, navigate to the **Business Travel** process.
2. After the **Notify customer travel confirmed** service task, add a **user task** and name it "Business travel feedback".
3. Update the user task properties:
    - Navigate to **Properties** > **Form** > **Type**.
    - Select "Camunda Form (embedded)".
    - In the **Form JSON configuration** field, enter the following code:

```
{
  "executionPlatform": "Camunda Cloud",
  "executionPlatformVersion": "8.4.5",
  "exporter": {
    "name": "Camunda Web Modeler",
    "version": "917d9a1"
  },
  "schemaVersion": 10,
  "components": [
    {
      "text": "### Business Travel Feedback",
      "type": "text",
      "layout": {
        "row": "row_1",
        "col": "auto"
      },
      "id": "Heading_1"
    },
    {
      "label": "Destination",
      "type": "textfield",
      "layout": {
        "row": "Row_10ze6m1",
        "columns": null
      },
      "id": "Field_1118l0w",
```

```
      "key": "travelDestination",
      "description": "Business travel destination",
      "readonly": false
    },
    {
      "subtype": "date",
      "dateLabel": "Date",
      "label": "Date time",
      "type": "datetime",
      "layout": {
        "row": "Row_10ze6m1",
        "columns": null
      },
      "id": "Field_1s4wi54",
      "key": "travelDate"
    },
    {
      "text": "##### Flight Details",
      "type": "text",
      "layout": {
        "row": "row_2",
        "col": "auto"
      },
      "id": "Subheading_1"
    },
    {
      "text": "##### Hotel Details",
      "type": "text",
      "layout": {
        "row": "row_2",
        "col": "auto"
      },
      "id": "Subheading_2"
    },
    {
      "values": [
        {
          "value": "LH 1074",
          "label": "LH 1074"
        },
        {
          "value": "LH 1036",
          "label": "LH 1036"
        },
        {
          "value": "LH 1089",
          "label": "LH 1089"
        }
      ],
      "type": "select",
      "layout": {
        "row": "row_3",
        "col": "auto"
      },
      "id": "Select_1",
```

```
      "label": "Start Flight",
      "key": "travelFlight",
      "searchable": false
    },
    {
      "values": [
        {
          "value": "Hotel Résidence Montebello",
          "label": "Hotel Résidence Montebello"
        },
        {
          "value": "Hotel Park Lane",
          "label": "Hotel Park Lane"
        },
        {
          "value": "Hotel Rivoli",
          "label": "Hotel Rivoli"
        }
      ],
      "type": "select",
      "layout": {
        "row": "row_3",
        "col": "auto"
      },
      "id": "Select_2",
      "label": "Hotel",
      "key": "travelHotel"
    },
    {
      "text": "---",
      "label": "Text view",
      "type": "text",
      "layout": {
        "row": "Row_1ens538",
        "columns": null
      },
      "id": "Field_0991vfs"
    },
    {
      "label": "How was your travel? (optional)",
      "type": "textarea",
      "layout": {
        "row": "Row_0d9voh8",
        "columns": null
      },
      "id": "Field_1xxtb8x",
      "key": "travelFeedback"
    },
    {
      "label": "Reimbursement needed?",
      "type": "checkbox",
      "layout": {
        "row": "Row_0nnq9xe",
        "columns": null
      },
```

```
      "id": "Field_0au7wgc",
      "key": "reimbursementNeeded",
      "defaultValue": true
    }
  ],
  "generated": true,
  "type": "default",
  "id": "select-flight-and-hotel-01n9xjw"
}
```

Info

In this form you will check whether the reimbursement is needed using the **reimbursementNeeded** Boolean variable.

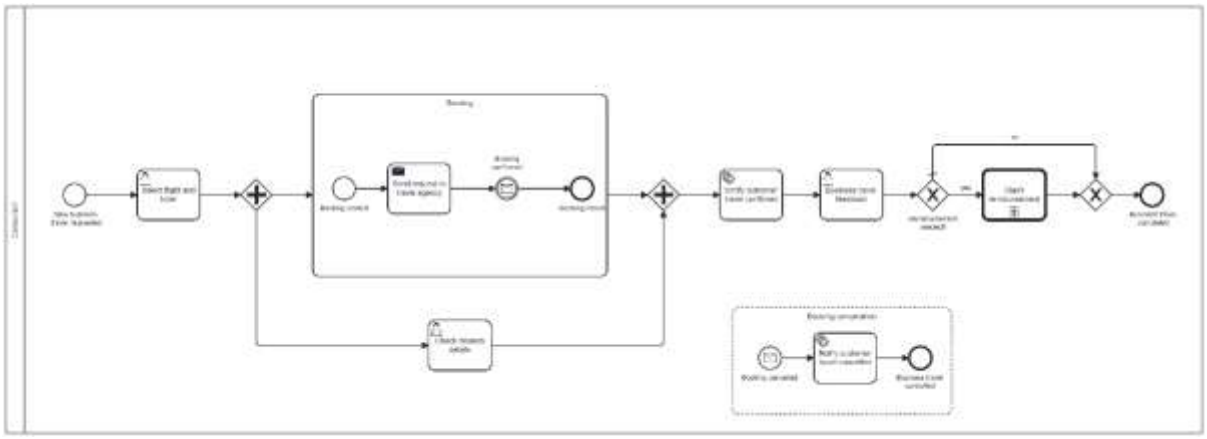
Exclusive gateways

1. After the **Business travel feedback** user task, add an **exclusive gateway** and name it "Reimbursement needed?".
2. After that exclusive gateway, add another exclusive gateway. This will be used as a **process flow merge**.
3. Label the sequence flow between the exclusive gateways "yes".
    - In the FEEL expression field, name the condition expression "reimbursementNeeded".
4. Add another **sequence flow** between the exclusive gateways.
5. Label this second sequence flow "no", and mark it as the **default flow**.

Call activity

1. Add a **call activity** to the **yes** process flow and name it "Claim reimbursement".
2. Update the call activity properties:
    - Navigate to **Properties** > **Called element**.
    - In the **Process ID** field, enter "ReimbursementClaimProcess".
    - Under **Input propagation**, ensure that the switch is toggled to **On**.

The process will now look like this:



Business Travel process with reimbursement claim improvement

Deploy the Business Travel process

1. Deploy the updated version of the Business Travel process.

Process improved!
You improved the Business Travel process by adding a user task to gather feedback from the consultant, and adding a call activity to the Reimburse Claim process to handle cases where expense reimbursements are needed.

Run & test process

Deploy the process

1. In Camunda 8 Web Modeler, navigate to the **Business Travel** process and deploy it to your main cluster.

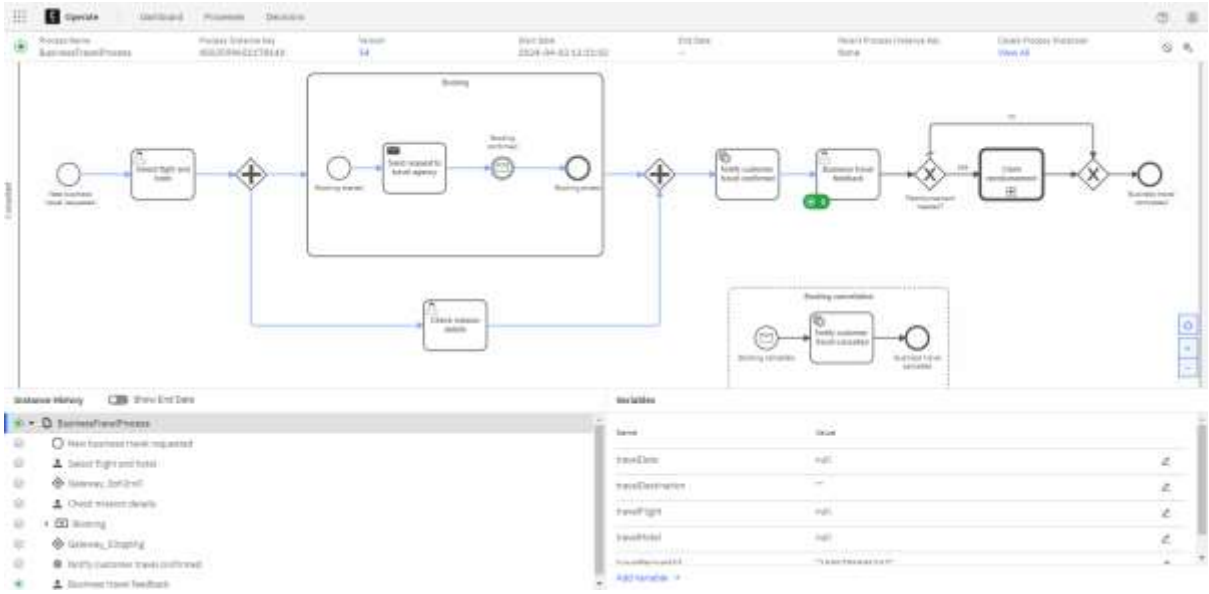
Run and test

1. Repeat all the steps from the **Process communication - Run & test** lesson to move the process instance token to the **Business travel feedback** user task:

    - Run both the `BusinessTravelApplication.java` and `TravelAgencyApplication.java` classes.
    - Run a new Business Travel process instance from the Camunda 8 **Modeler** using the following JSON as an input:

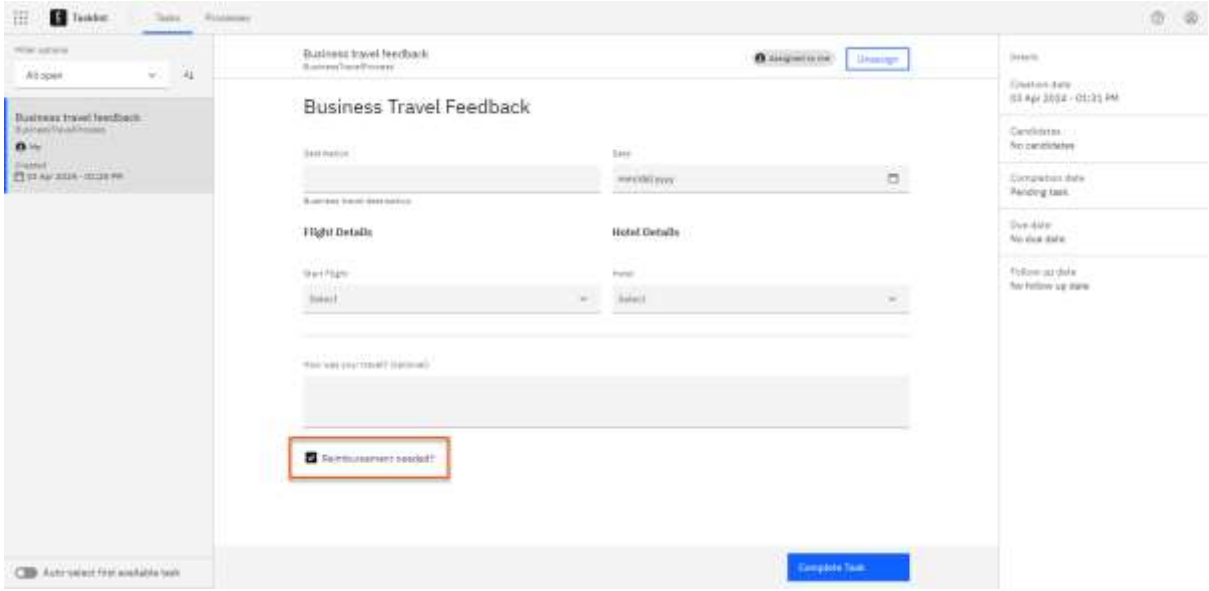
```
{
  "travelDate": "2024-11-06",
  "travelDestination": "Paris",
  "travelFlight": "LH 1074",
  "travelHotel": "Hotel Park Lane",
  "travelRequestId": "1695286596244"
}
```

    - ...
2. Run the `ReimbursementClaim.java` class.
3. In Camunda 8 **Operate**, verify the Business Travel process instance is waiting at the **Business travel feedback** user task.



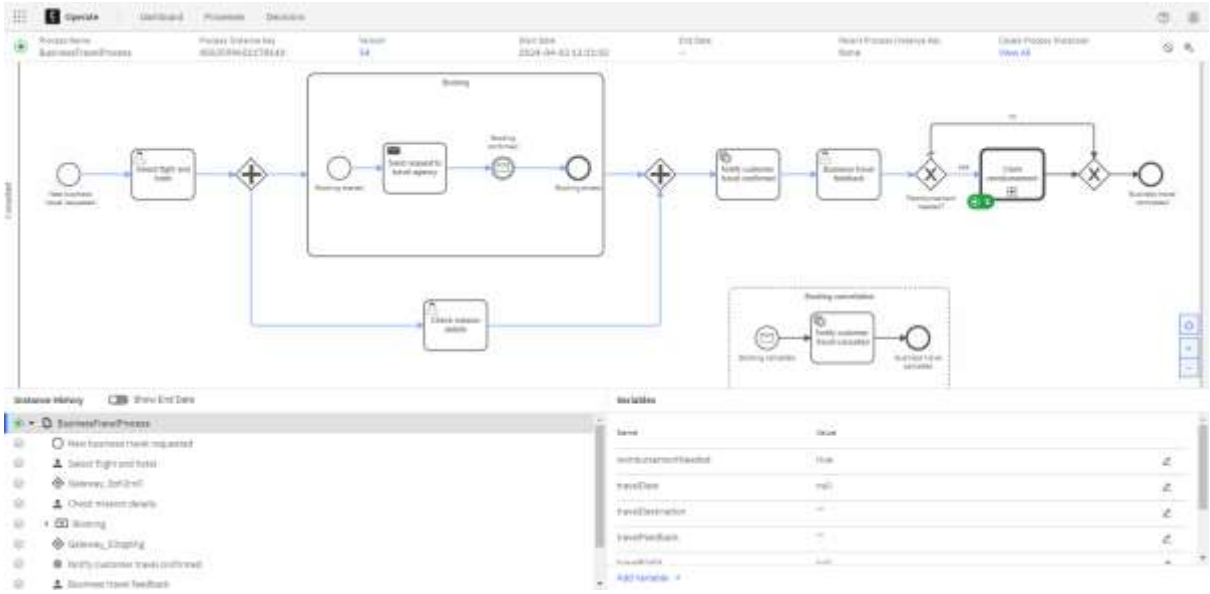
Business Travel process with feedback user task

4. In Camunda 8 **Tasklist**, assign the **Business travel feedback** task to yourself.
5. Verify that "Reimbursement needed?" is selected, and then complete the task.



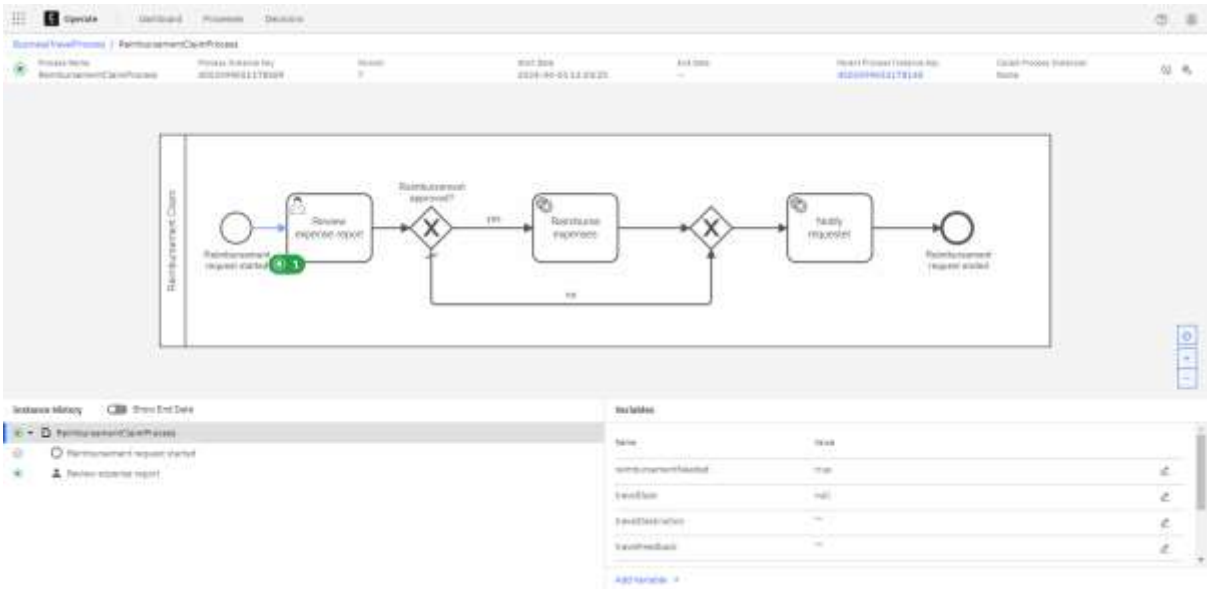
Business Travel process with feedback form

6. Navigate to Camunda 8 **Operate** and verify that there are now two instances:
    - The **BusinessTravelProcess** instance is now on the **Claim reimbursement** call activity.



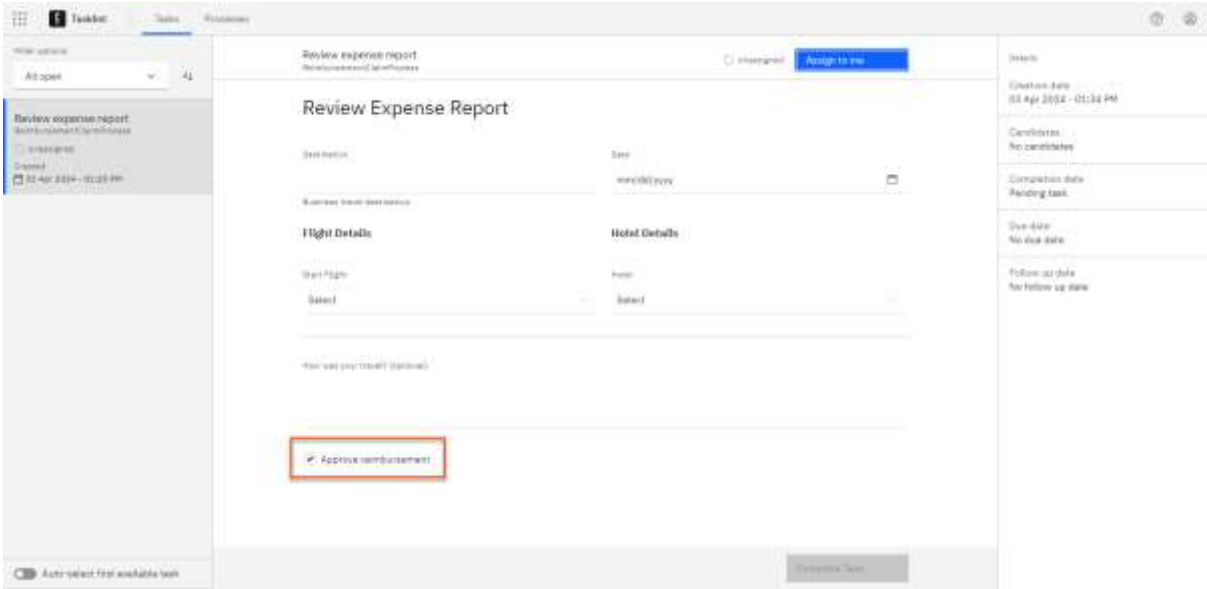
Business Travel process with claim reimbursement call activity

- The **ReimbursementClaimProcess** instance is on the **Review expense report** user task.



Reimbursement Claim process with user task to review expense report

7. In Camunda 8 **Tasklist**, assign the **Review expense report** task to yourself.
8. Verify that "Approve reimbursement" is selected, and then complete the task.



Business Travel process with Review Expense Report form

9. Navigate to the IDE console to review the last printed messages from the `Reimbursement Claim`:

```
Connected to Cluster 1: TopologyImpl{brokers=[BrokerInfoImpl{nodeId=1,
host='zeebe-1.zeebe-broker-service.xxx-zeebe.svc.cluster.local', port=26501,
version=8.5.0, partitions=[PartitionInfoImpl{partitionId=1, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=2, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=3, role=LEADER,
health=HEALTHY}]}, BrokerInfoImpl{nodeId=0, host='zeebe-0.zeebe-broker-
service.xxx-zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]},
BrokerInfoImpl{nodeId=2, host='zeebe-2.zeebe-broker-service.xxx-
zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]}],
clusterSize=3, partitionsCount=3, replicationFactor=3, gatewayVersion=8.5.0}
1695286596244 Reimbursement Request: Expenses reimbursed
1695286596244 Reimbursement Request: Consultant notification sent
```

All process instances are now completed.

Call Reimbursement Claim process

Congratulations! You successfully called the Reimbursement Claim process using a call activity.

Understand business requirement

Analysis

Last but not least, any changes to the Travel Policy must be communicated to the consultant.

Business Travel process

The **Business Travel** process handles the booking through the **Booking** expanded subprocess. If the travel policy is updated during the booking process, a consultant who is at that step in the process should be notified.

Conclusion

You can improve the Business Travel process by adding a **non-interrupting signal boundary event** to the **Booking** expanded subprocess, so that if the travel policy changes, the consultant receives a notification.

Reminder

Signal events are good for broadcast communication. Remember that this type of notification will apply to *any* consultant who is performing business travel.

In the next lesson you will implement these changes.

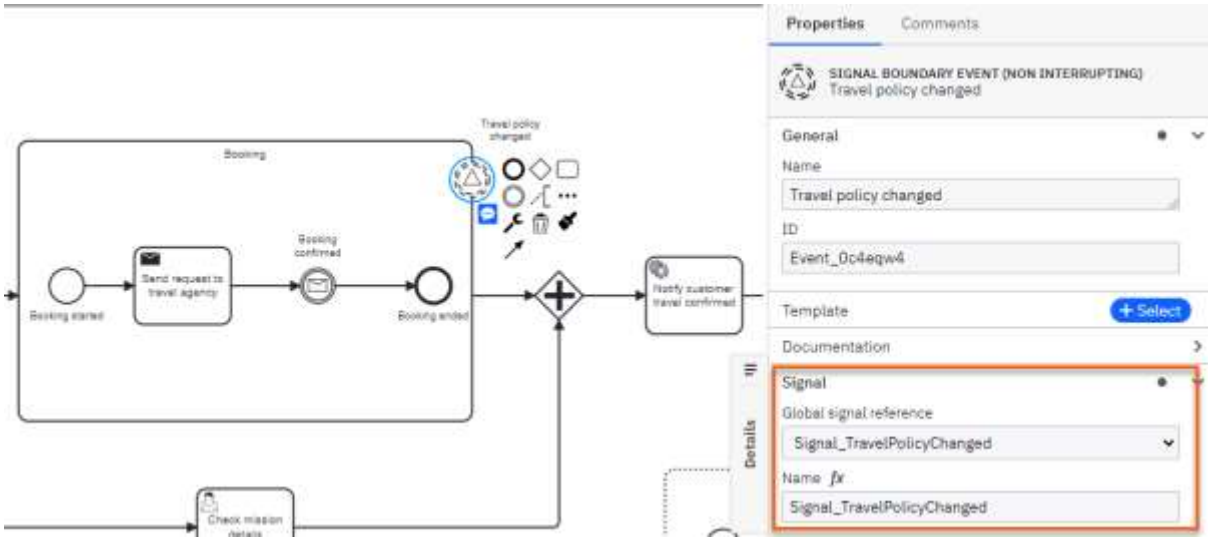
Implement adjustments

Implement the Business Travel process

Update the Booking expanded subprocess

1. In Camunda 8 Web Modeler, navigate to the **Business Travel** process.
2. In the **Booking** expanded subprocess, add a **non-interrupting signal boundary event** and name it "Travel policy changed".
3. Update the signal boundary event properties:

- Navigate to **Properties** > **Signal**.
- Create a new **Global signal reference**.
- In the **Name** field, enter "Signal_TravelPolicyChanged".



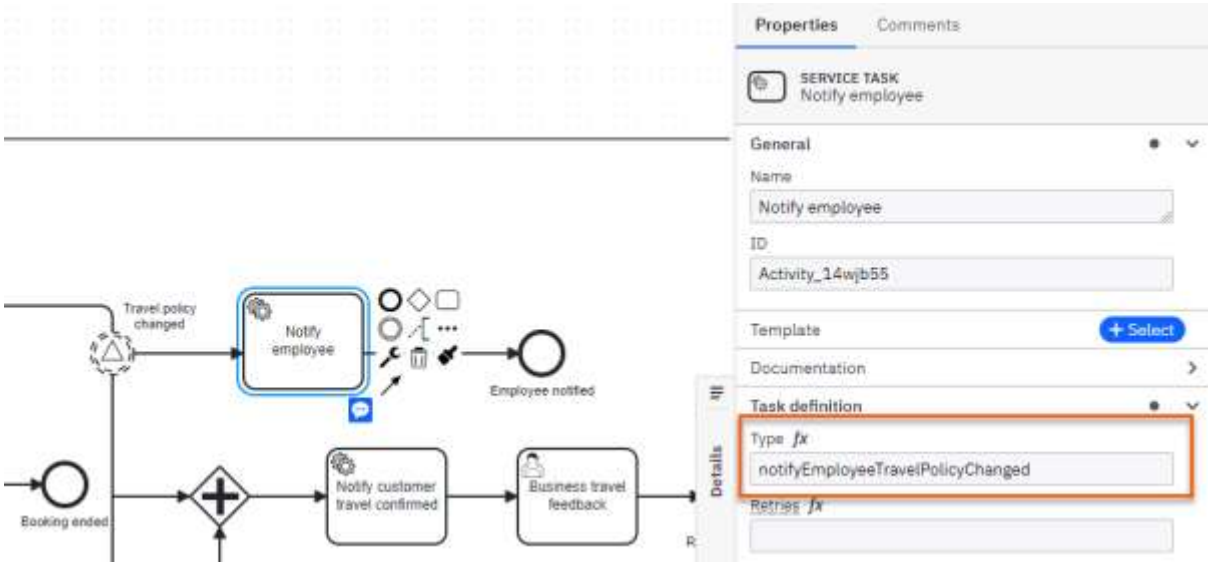
Business Travel process with a non-interrupting signal boundary event

Service task

1. After the non-interrupting signal event, add a **service task** and name it "Notify employee".
2. Update the service task properties:

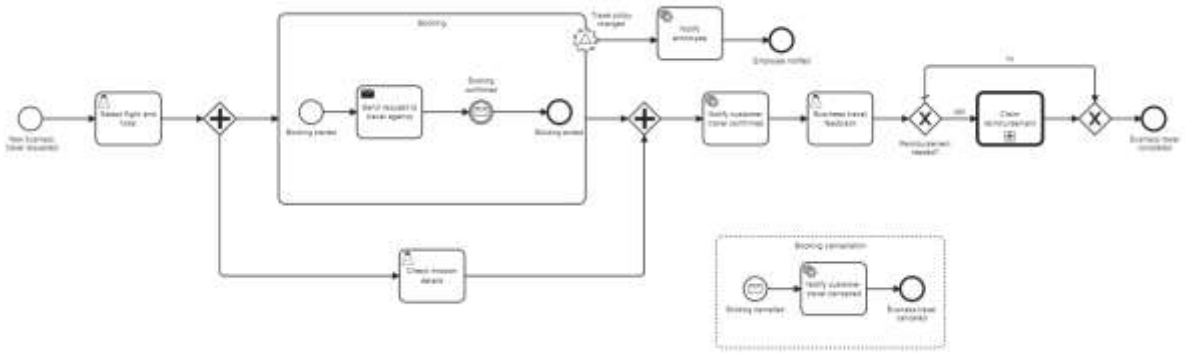
- Navigate to **Properties** > **Task definition**.
- In the **Type** field, enter "notifyEmployeeTravelPolicyChanged".

3. Add an end event and name it "Employee notified".



Business Travel process with service task to notify employee

The process will now look like this:



Business Travel process

Process improved!

You improved the Business Travel process with a non-interrupting signal event at the boundary of the Booking expanded subprocess to notify the employee whenever the Travel policy changes.

Run & test process

Deploy the process

1. In Camunda 8 Web Modeler, navigate to the **Business Travel** process and deploy it to your main cluster.

Run and test

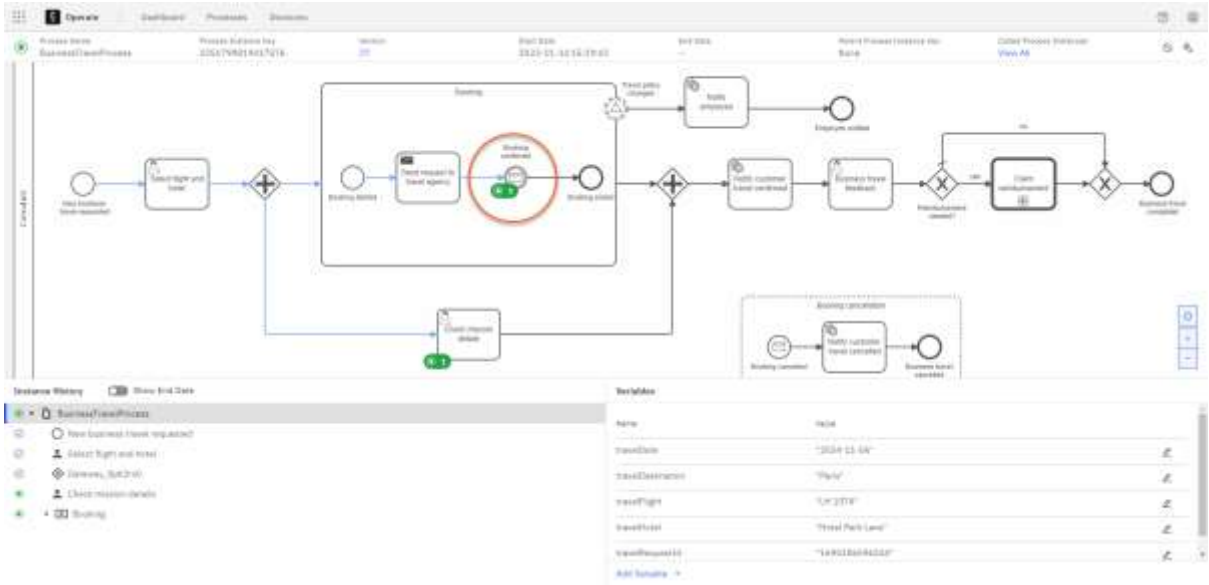
1. Repeat all the steps from the **Process communication - Run & test** lesson to move the process instance token to the **Booking confirmed** catch event in the **Booking** expanded subprocess:

    - Run both the `BusinessTravelApplication.java` and `TravelAgencyApplication.java` classes.
    - Run a new Business Travel process instance from the Camunda 8 **Modeler** using the following JSON as an input:

```
{
  "travelDate": "2024-11-06",
  "travelDestination": "Paris",
  "travelFlight": "LH 1074",
  "travelHotel": "Hotel Park Lane",
  "travelRequestId": "1695286596244"
}
```

    - ...

2. In Camunda 8 **Operate**, verify the Business Travel process instance is waiting at the **Booking confirmed** catch event.



Business Travel process waiting at booking confirmed catch event

3. From a **Terminal**, use the **Zeebe API (CLI Client)** to send the signal executing this command:

```
zbctl broadcast signal Signal_TravelPolicyChanged
```

The result will be a key:

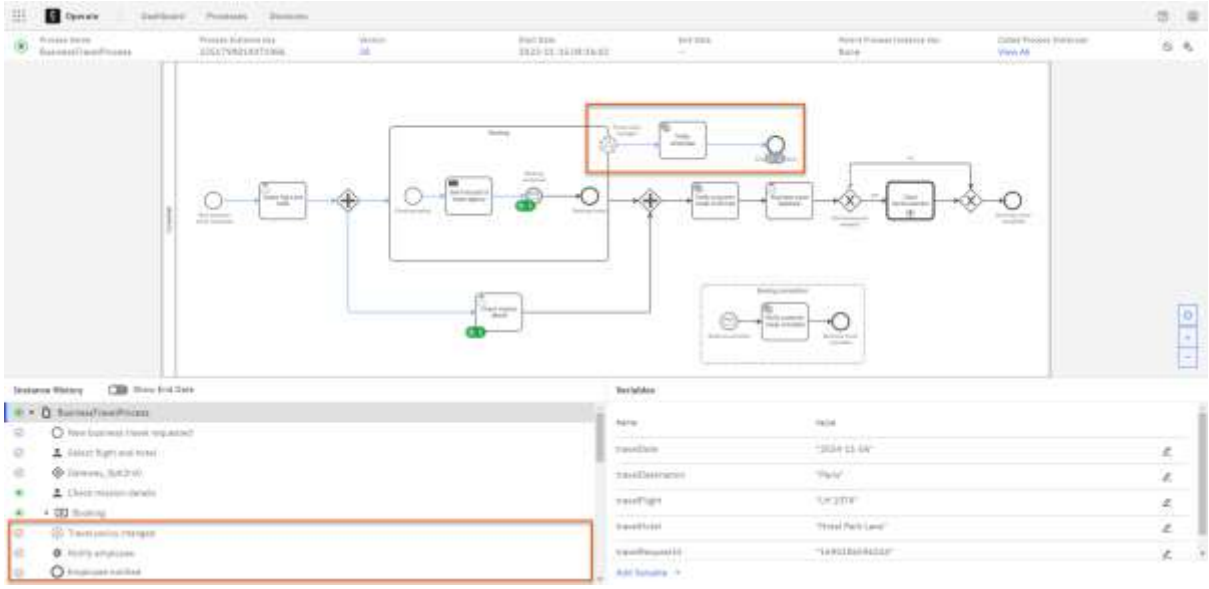
```
{
  "key":  "2251799819418641"
}
```

4. Now, check your IDE console to review the last printed messages from the `Business Travel` application:

```
Connected to Cluster 1: TopologyImpl{brokers=[BrokerInfoImpl{nodeId=1,
host='zeebe-1.zeebe-broker-service.xxx-zeebe.svc.cluster.local', port=26501,
version=8.5.0, partitions=[PartitionInfoImpl{partitionId=1, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=2, role=LEADER,
health=HEALTHY}, PartitionInfoImpl{partitionId=3, role=LEADER,
health=HEALTHY}]}, BrokerInfoImpl{nodeId=0, host='zeebe-0.zeebe-broker-
service.xxx-zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]},
BrokerInfoImpl{nodeId=2, host='zeebe-2.zeebe-broker-service.xxx-
zeebe.svc.cluster.local', port=26501, version=8.5.0,
partitions=[PartitionInfoImpl{partitionId=1, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=2, role=FOLLOWER, health=HEALTHY},
PartitionInfoImpl{partitionId=3, role=FOLLOWER, health=HEALTHY}]}],
clusterSize=3, partitionsCount=3, replicationFactor=3, gatewayVersion=8.5.0}
```

```
1695286596244 Travel Policy changed
```

5. In Camunda 8 **Operate**, verify that the Business Travel process instance has received the signal event, notified the employee, and finished the process.



Business Travel process with travel policy changed and employee notified

Multiple instances

You can create additional process instances and send the signal again to verify that they will call catch the signal and notify the corresponding consultants.

Notify Travel Policy changed

Congratulations! You successfully sent a signal to the Booking extended subprocess to notify the consultant if the Travel Policy was changed.

Example solution

If you encountered any problems during this course or were unable to complete the process and decision configurations, you can compare your solution with the example solution, located in GitHub:

- Solution - Business Travel application